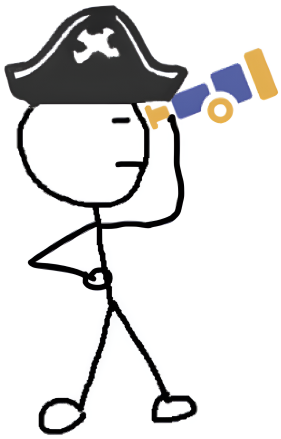
Evaluating OpenTelemetry's Impact on Performance in Microservice Architectures

Frode Sandberg



Spring 2024
Examiner: Henrik Björklund
Supervisor: Alexandre Bartel
External Supervisor: Johannes Lindgren

Degree Project in Computing Science and Engineering, 30 credits
Master of Science Programme in Computing Science and Engineering, 300 credits

External Partner: Nasdaq Technology AB

Abstract

Observability grants engineers the ability to comprehensively understand their systems by generating, exporting, and analyzing telemetry; logs, metrics, and traces. This is especially important for microservice-based systems as their distributed nature makes them notoriously difficult to debug. OpenTelemetry is an observability tool which has steadily risen in popularity since its inception in 2019, its purpose is to generate and export telemetry. This thesis investigates the performance impact of implementing OpenTelemetry in a microservice-based system as performance overhead can be a key concern for organizations.

The experiments conducted in this thesis demonstrate that OpenTelemetry impacts performance, for instance, in terms of CPU overhead, which reached 42% under certain conditions. Compared to equivalent tools, OpenTelemetry performed better for tracing but slightly worse for metrics. The latter result might stem from the OpenTelemetry agent not being designed to be used solely for metric instrumentation. Furthermore, the results indicate that manually generating traces with OpenTelemetry incurs less overhead compared to using the agent for automatic trace instrumentation. This difference is most notable in terms of CPU overhead, where automatic instrumentation incurred roughly twice as much overhead. Finally, the experiments demonstrated that batching and head-based sampling can significantly reduce OpenTelemetry's performance impact. This reduction is most notable in terms of CPU and latency overhead, which decreased to 3.6% and 3.4% respectively.

Acknowledgments

I would like to acknowledge and sincerely thank the following individuals and inanimate object for their contributions to this thesis.

- **Alexandre Bartel:** For helping me with the general thesis structure and inspiring the observability pirate with the exquisite art he has had on display in his office.
- **Johannes Lindgren:** For letting me borrow his powerful brain weekly and pushing the thesis forward with ideas and discussion.
- **Niclas Dimberg:** For extensively proofreading the thesis, and helping me arrive at this thesis subject in the first place.
- **The Nasdaq Coffee Machine:** For consistently turning beans into thoughts just about coherent enough for me to complete this thesis.

Contents

1 Introduction

As distributed systems become larger and more complex, it becomes harder to gain an understanding of their behavior, performance, and health. Observability is a rapidly growing area within distributed systems. Its primary aim is to tackle the complexity challenge by generating and analyzing telemetry data, including metrics, logs, and traces from the system. An interesting tool for this purpose is OpenTelemetry. It can be used to instrument, generate, collect, and export telemetry data. OpenTelemetry is a Cloud Native Computing Foundation (CNCF) project that was founded in 2019. It was moved to the incubating maturity level as recently as August 2021 and has since continued to gain momentum. Today, it is the second most active CNCF project; only Kubernetes is more active. Some suggest it is becoming the industry standard for instrumentation.

One key factor to consider when implementing observability is how the system's performance will be impacted. An organization is unlikely to introduce a new tool in its system if it has a large impact on, for instance: the request latency, CPU, memory, or network usage. Microservice architectures are particularly interesting as their decentralized and loosely coupled nature makes them especially difficult to debug without the use of observability.

The goal of this thesis is to examine the performance impact of implementing OpenTelemetry in a microservice system. This will be achieved by answering the following questions:

1. How is performance impacted by implementing OpenTelemetry for traces/metrics?
2. How does the above compare to other prevalent tools?
3. How effective are strategies such as batching or sampling at reducing the performance impact of OpenTelemetry for tracing?

The subject of this thesis is based on a proposal from Nasdaq and a future work suggestion by Kristoffer Gilliusson [1].

2 Background

This section explains the key theoretical concepts the thesis revolves around and gives an introduction to OpenTelemetry and its components.

2.1 Microservices

A system can be considered to utilize a microservice architecture if it consists of a set of loosely coupled services. In this context, a service can be considered a small application with a clear, cohesive set of responsibilities [2]. A service may interact with other services, typically either by using message brokers (such as RabbitMQ or Kafka) or lightweight

protocols such as REST or gRPC [2]. The fact that communication is facilitated with technology-agnostic protocols enables heterogeneity in the system [3]. This means a microservice can be implemented in the language or framework best suited for its specific task.

There are many advantages to using a microservice architecture compared to a more traditional monolithic approach. Usually, services are reasonably small and have specific responsibilities, making them easier for developers to understand and maintain [4]. Since each service operates as an independent unit, teams can work concurrently on different services without interfering with each other's work [4]. Another advantage is that services can be easily spread out across multiple machines and also scaled independently, allowing for greater flexibility when handling complex loads [4].

One common pattern that microservice architectures use is the "API gateway pattern" which lets clients access the system through a single service responsible for routing requests through the entire system [2]. The single point of entry simplifies the client implementation as it only has to interact with one service. It also reduces the need for other services to consider security, since the gateway can handle authentication before sending requests to other services [5]. Figure 1 illustrates an implementation of this pattern.

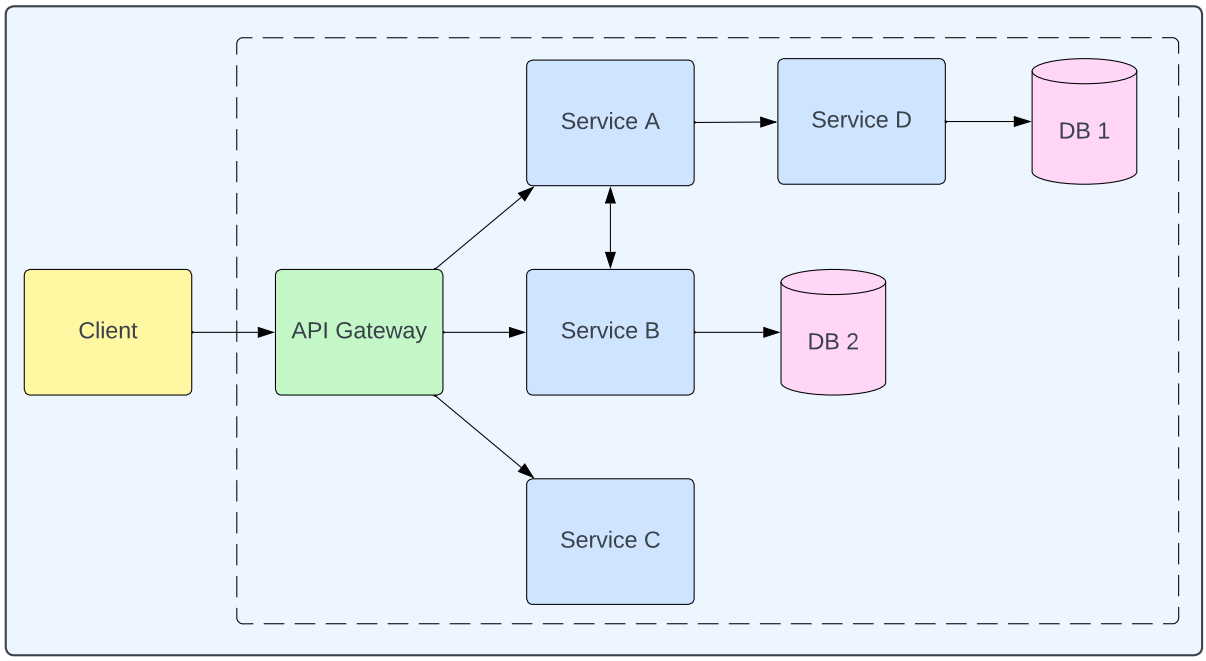


Figure 1: Example of a client interacting with a system that implements a microservice architecture using the API gateway pattern.

One key issue with microservices is that it can be complicated to understand and diagnose problems, e.g. failed requests or high latency. This can be attributed to complex cloud-based runtime environments where systems can be distributed over multiple machines as well as complex chains of dependencies between multiple services [6]. If a request to `Service B` in Figure 1 is returned with higher than acceptable latency, how can the source of the problem be determined? The problem could be in `Service B`, but it could also be in any of the components that it interacted with to fulfill the request. The fact that

there can be multiple replicas of each component further complicates the problem. When such a problem occurs in a system, the goal is to identify which services the request went through, what each service did, and which service(s) caused the problem [3]. To achieve this, a comprehensive understanding of the behavior of the entire system is needed. This is what observability aims to achieve.

2.2 Observability

Observability as a concept is not unique to computer science. The term was coined in the year 1960 in the area of control theory to describe mathematical control systems. In control theory it is defined as "a measure of how well internal states of a system can be inferred from knowledge of its external outputs" [7]. The authors of the book *Observability Engineering* extend this definition to software systems. They mention that four conditions must be fulfilled for a software application to be considered to have observability [8]:

1. The inner workings of the application can be understood.
2. It is possible to understand any state the system can get into.
3. The things above can be understood, solely by observing the system with external tools.
4. Any state can be understood, no matter how extreme or unusual.

Once observability has been obtained it should be possible to debug any issues that occur in the system, no matter how novel or bizarre, without needing to add any new code to the system [8]. In practice, this is achieved by instrumenting the system so that it emits telemetry data which can be stored and analyzed by an observability backend. There are three core types of telemetry data: metrics, logs, and traces. Sometimes they are referred to as the "three pillars of observability".

2.2.1 Metrics

A metric is a numerical representation of data that typically falls into two categories. The first is data that is already numeric, i.e. temperature or bytes of memory currently in use. The second type is data that can be distilled or aggregated into numbers such as the number of requests served, tasks completed or errors observed [9].

Most tools for metric instrumentation will have implemented three core metric types. Firstly there is the *gauge*, a metric representing a single numerical value that can go up or down arbitrarily. The *counter* is another type that represents a numerical value that is strictly increasing. Lastly, there is the *histogram* type which samples observations and puts them into buckets [9]. A gauge would belong to the first category mentioned, while a counter would belong to the second category. A histogram could be considered a combination of the two categories as it aggregates numeric values.

Metrics are typically captured as time series data, enabling the analysis of changes in metrics over time [10]. There are two ways metrics are commonly used. The first is real-time monitoring and alerting, as in engineers monitoring the current system health or getting alerts when certain metrics pass certain thresholds. The second use case is spotting and analyzing trends for long-term planning or gathering insights after an incident [9].

2.2.2 Logs

A log can be defined as a piece of text that is generated by a system when a specific part of code executes; logs are records of specific events [11]. The most traditional form of logging is the act of writing print statements to be displayed in the console. This is perfectly viable for many systems, but something more sophisticated is needed for more complex systems consisting of multiple services spread out over several machines.

In the context of observability, logging typically refers to *centralized* logging where logs from all components of a system are sent to an observability backend where they can be searched for and analyzed [10]. Traditionally logs have been unstructured, meaning they were created to be read by humans rather than computers [8]. Unstructured logs might look like this:

```
13:37:01 user "op" requests authentication
13:37:03 successfully authenticated user "op"
13:37:07 processing request from "op"
```

As these logs lack uniform formatting they are difficult for computers to parse, which is problematic when there are too many logs for humans to read. For computers to search through and analyze large amounts of logs, they must be structured [8]. However, there is no universal standard for how logs should be structured. Observability backends that offer centralized logging can typically parse multiple standards [12]. Some alternatives include XML, CSV, Syslog, Common Log Format (CLF), and JSON [12]. For example, the logs above could be structured in JSON format like this:

```
{"time":"13:37:01", "user":"op", "message":"authentication requested"}
{"time":"13:37:04", "user":"op", "message":"authentication successful"}
{"time":"13:37:07", "user":"op", "message":"processing request"}
```

For logs to be useful, particularly in a distributed system, they typically have to contain far more information than in the example above. For example; process ID, thread ID, IP addresses, severity level (such as info, debug, warning, error or alert), and so on [12]. Knowing which logs were generated by the same request could also be useful. This is something that tracing enables.

2.2.3 Traces

Tracing or "distributed tracing", sometimes abbreviated "DT", is the newest of the three pillars. A trace grants developers the ability to inspect the journey of a single request, like the one in Figure 2, through the entire system. This is achieved by generating event data at various points in the system and tying them together with a unique identifier that gets

propagated across all components necessary for completing the request [13]. Therefore, every component can associate their event data with the originating request.

Google's Dapper is one of the earliest large-scale implementations of tracing. In 2010 the Dapper researchers released an article describing what they had learned from using Dapper internally for two years [14]. This paper is largely considered to be the foundation of modern tracing [13]. Formally, the researchers behind Dapper define a trace as *a tree of spans* [14]. More recently the definition of a trace has changed to be a *directed acyclic graph (DAG) of spans*, OpenTracing [15] and OpenTelemetry [16] are examples of projects that use this definition. A tree is a directed acyclic graph with the extra limitation that every child (except for the root) must have exactly one parent. In other words, the modern definition is *looser* as it allows for a span to have multiple parent spans, which would represent multiple requests getting merged into one single request. This property is used when batching requests [10].

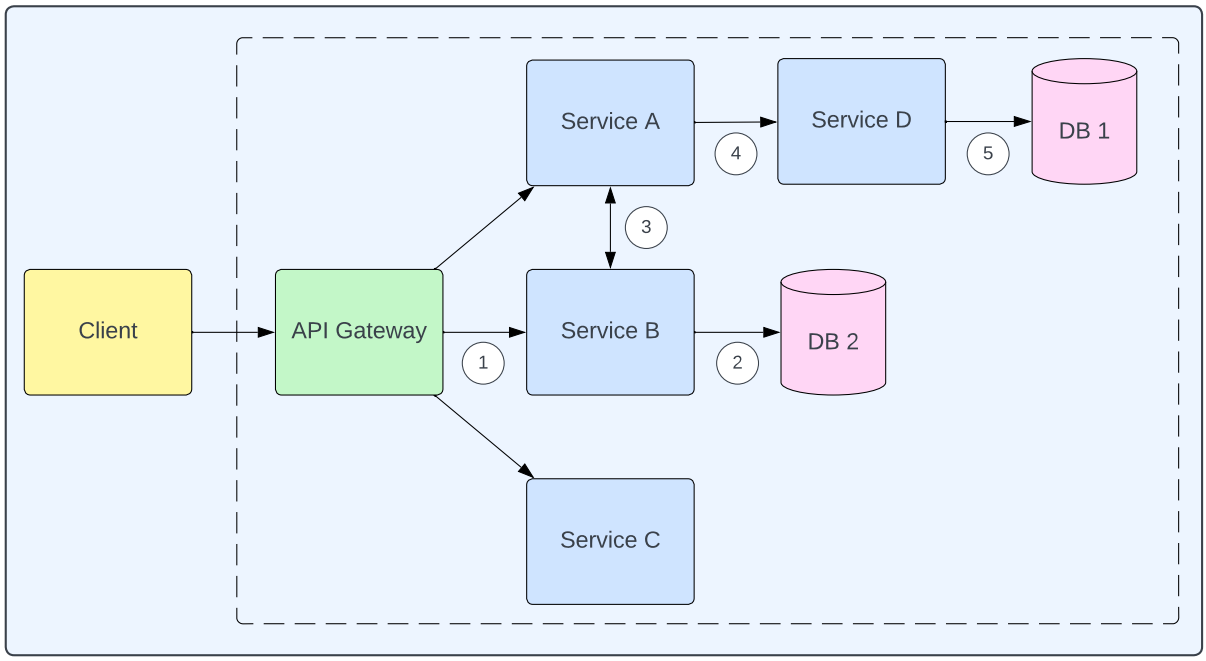


Figure 2: An example of a client request making its journey through a system.

A span is a data structure that represents a unit of work, it *must* contain the following information [8]:

- **Trace ID**: A unique identifier for the trace which is created by the root span and propagated to the children spans in the trace.
- **Span ID**: A unique identifier for all individual spans created.
- **Parent ID**: The Span ID of the parent (tree definition) or parents (DAG definition) that created this span. Absent if this is the root span.
- **Start time**: Timestamp for when the work started.
- **End time**: Timestamp for when the work ended.

Most actual tracing implementations put additional information in the spans, for example [13]:

- **Description**: A description of the work done.
- **Service name**: The name of the service that created the span.
- **Process**: Information about the application or process that carried out the work.
- **Tags**: Any additional information about the work.

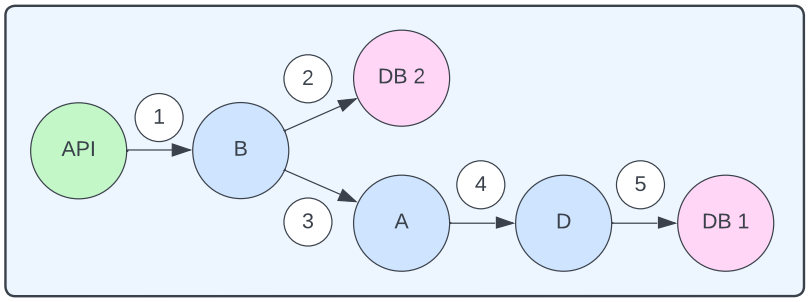


Figure 3: The trace generated in Figure 2 represented as a directed acyclic graph (DAG) of spans.

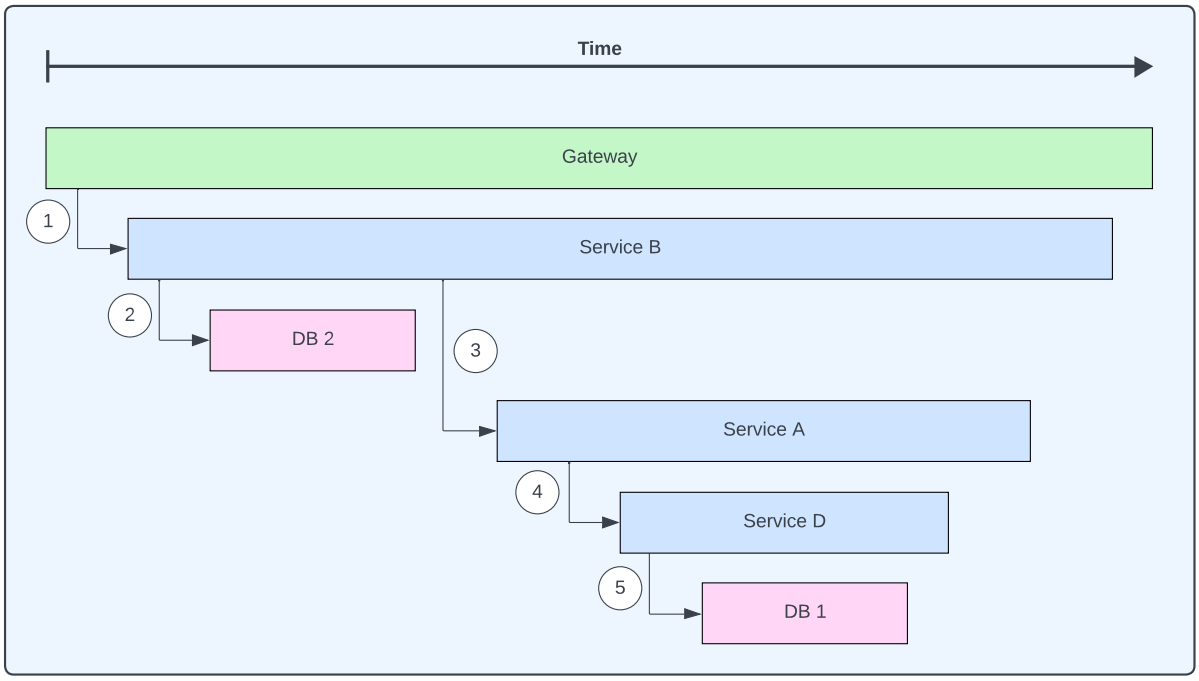


Figure 4: The trace generated in Figure 2 represented as a Gantt chart.

Figure 3 illustrates the trace generated by the request in Figure 2 as a DAG (that also happens to be a tree) of spans. For the sake of not cluttering the illustration, every service creates one span when they get the request. It is important to note that a service could

generate multiple spans for the same request. For example, one might want to separate the work done in different methods within the same service.

The representation in Figure 3 only captures the correlation between different spans. Since the start and end times are known for every span, the temporal relationship between spans in a trace can be visualized. This is commonly done by using Gantt charts, Figure 4 showcases what the Gantt representation of the trace from Figure 2 might look like.

2.2.4 Observability Backends

The responsibility of storing and analyzing the telemetry generated by different system components typically falls on an external component. Commonly, these are referred to as "observability backends". There is an abundance of observability backends, both proprietary and open-source. Some specialize in specific types of telemetry, others can handle all three pillars.

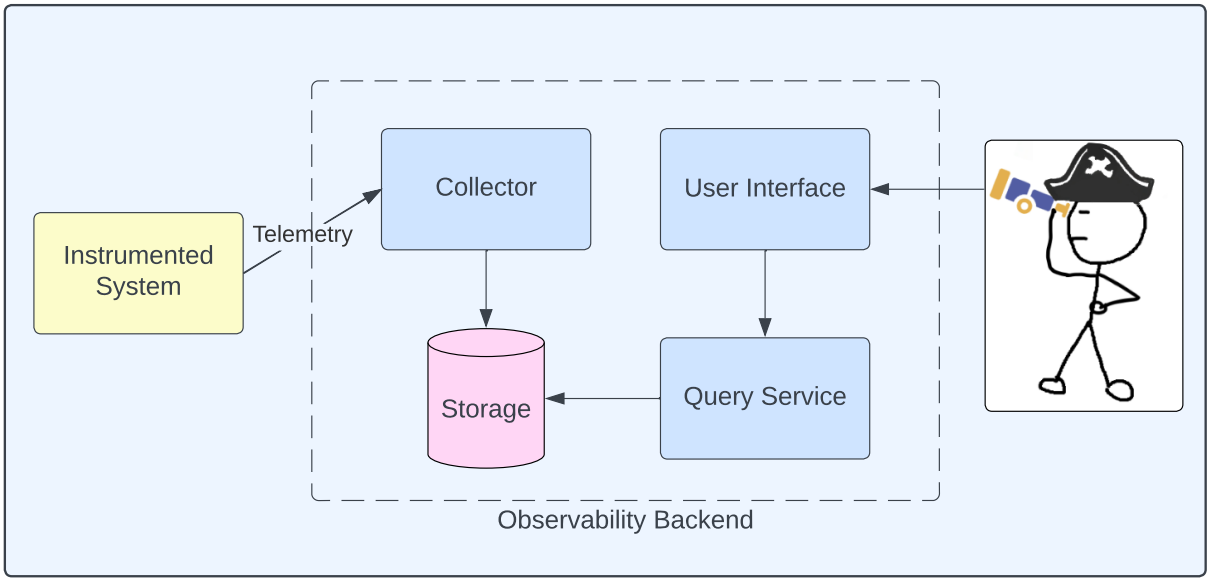


Figure 5: The observability pirate examining the telemetry generated by his system through a generic observability backend. Based on the architectures of SigNoz [17], Jaeger [18] and Zipkin [19].

Figure 5 exemplifies how a generic observability backend might work. It is based on the architecture of three open-source examples; SigNoz [17], Jaeger [18], and Zipkin [19]. The figure illustrates how an observability backend can be thought of as having four main components. The *collector* is responsible for receiving the telemetry and sending it to the *storage* component. The observability backend can be interacted with through the *user interface* which sends requests to the *query service* that is responsible for extracting the requested data from the storage.

2.3 Performance Overhead

Few things in life come free, and observability is no exception. While there are many benefits to making a system observable, there are also costs, particularly in terms of performance overhead.

The generation of telemetry data requires computation, the data will then be stored in memory. At some point, the data will be serialized and sent over the network to an observability backend where the data is deserialized and stored. Tracing also requires that certain information is propagated with requests. This is known as context propagation [20]. For example, the next service in the request chain needs to know which trace its spans belong to and which span ID is the parent ID of their first span. When this context travels between services, it also requires serialization, network usage, and deserialization [20]. The actual observability backend itself also requires computation and memory to run. It also needs to store all the data it receives, most likely in some persistent storage. Figure 6 illustrates sources of overhead that stem from observability.

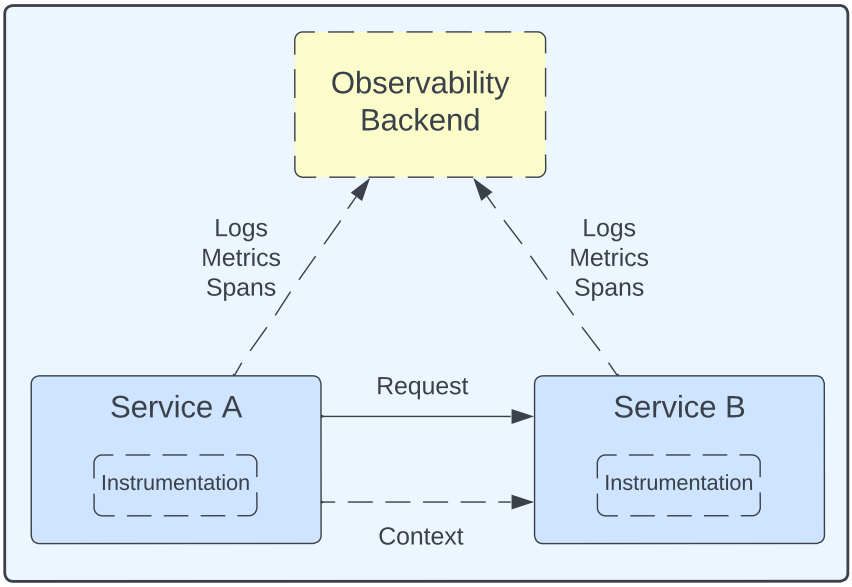


Figure 6: A request traveling from one service to another with observability. Dotted lines indicate sources of overhead.

After adding observability to a system, it is reasonable to expect that more persistent storage will be required and that there will be an increase in CPU, memory, and network usage. The added computation and resource usage can impact the system's end user, for example, in terms of added request latency. Fortunately, several trade-offs can be made to reduce the impact observability has on performance. Batching and sampling are two common ones.

2.3.1 Batching

When a service sends telemetry to an observability backend, the data has to travel over the network. However, additional information is required to, for example, know the data's

destination or check the data's validity [21]. This additional information is known as *protocol overhead*.

Rather than sending telemetry data individually to an observability backend, the data can be temporarily stored in memory and sent out in a batch with more telemetry data [3]. This reduces the protocol overhead since fewer overall messages are sent. However, there are two main drawbacks to this method. Firstly, more memory will be used to store the data as it accumulates. Secondly, there will be a greater delay between the time when the telemetry data is generated and when it is received by the observability backend. The latter could be problematic if real-time querying of the telemetry data is required.

2.3.2 Sampling

Sampling refers to reducing the amount of telemetry data that is generated, sent, and stored. It is a common practice to use sampling to minimize the overhead of logs [12] and traces [13]. However, sampling cannot be applied to metrics in the same way [13]. For instance, sampling a metric that counts requests would cause requests to be unaccounted for, making the metric lose its meaning [13]. When it comes to sampling traces, there are two common strategies: head-based and tail-based sampling.

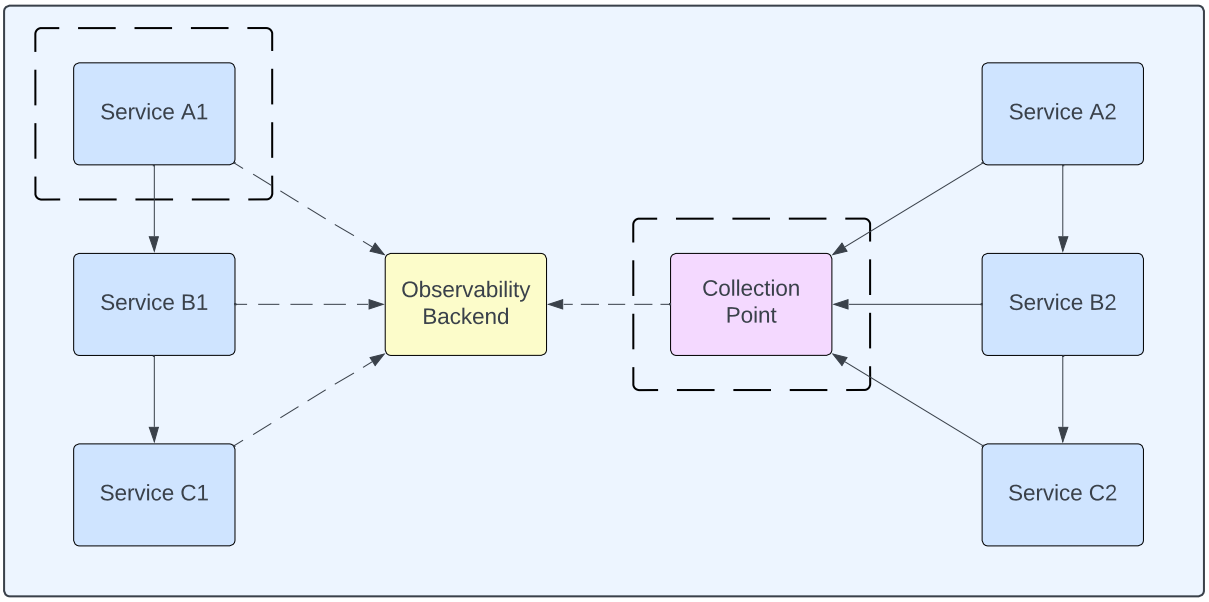


Figure 7: Services A1–C1 utilize head-based sampling whereas services A2–C2 utilize tail-based sampling. The dashed boxes show where the sampling decision is made, the dashed arrows represent sampled data.

Head-based sampling is a strategy where the root span decides whether or not to sample the trace. This decision is then propagated with the request so that all child spans know whether or not to sample the trace [13]. The services A1–C1 in Figure 7 use head-sampling. Typically, the sampling decision is made using a probabilistic approach, either by using a predetermined static probability or a dynamic probability that changes based on factors such as the current system load [14].

Tail-based sampling is a method of sampling where a decision is made after the entire trace is generated [13]. This decision is typically based on information found within the trace, such as errors or latency that exceed a certain threshold. The full trace must be collected at a central point between the services and the observability backend for the decision to be made [13]. The services A2–C2 in Figure 7 utilize tail-based sampling.

Both sampling strategies help reduce the number of traces stored. Head-based sampling also reduces the network and computational effort required to export non-sampled traces. Tail-based sampling is less powerful for reducing overhead but gives much more fine-grained control over what is sampled. Sampling can be done in multiple steps by combining the strategies.

2.4 OpenTelemetry

OpenTelemetry is an open-source observability project that provides open standards and tooling to independently instrument, collect, and transport the main types of telemetry data: metrics, logs, and traces [13]. The history and context of OpenTelemetry and the components that are most central to this thesis are explained in this section.

2.4.1 History and Context

Before OpenTelemetry, there were two popular open-source alternatives: OpenTracing (a CNCF project) and OpenCensus (a Google Open Source project). Both projects aimed to standardize the instrumentation of code and the transmission of telemetry data to observability backends [22, 23]. The active development of both projects made it unclear which to rely on as they both excelled at different things and were difficult to use together [13]. The maintainers of the respective projects decided that merging would be the best way of achieving their common goals. OpenTelemetry emerged in 2019 as an open-source project under CNCF as a result of their combined efforts [23]. Due to this historical context, the abbreviation of OpenTelemetry is OTel and *not* OT as that is the abbreviation for OpenTracing.

Since its inception, OpenTelemetry has quickly grown in popularity. CNCF, which hosts many of the most prominent open-source cloud native projects, promoted OpenTelemetry to the incubating maturity level in August 2021 [24], signaling that it is ready for production-grade systems. Many prominent organizations have adopted OpenTelemetry, for example; GitHub, Shopify, and eBay [25]. Based on CNCF's measurement of activity for their 173 projects, OpenTelemetry ranked as the second most active project within CNCF, surpassed only by Kubernetes [26]. Out of *all* open source projects, CNCF ranks OpenTelemetry as one of the top 30 most active [26].

Today, OpenTelemetry is considered the industry standard for instrumentation by many leading engineers in the observability space [8, 22]. A concrete example of this is how the popular open-source tracing tool Jaeger deprecated its tracing instrumentation SDK, urging users to use OpenTelemetry instead [27].

2.4.2 Documentation

The foundation of OpenTelemetry is the documentation. For metrics, logs, and traces, respectively, the following key documentation exists: a specification, a data model, and semantic conventions.

- **Specification:** These documents describe the requirements and expectations of all OpenTelemetry implementations. This ensures that users can expect a uniform experience regardless of the language used [13].
- **Data model:** Contains definitions of how the different types of telemetry data should be represented. It also explains what use cases the SDKs and APIs will support, as well as how the data should behave [13].
- **Semantic conventions:** Provides guidelines for what telemetry data should be reported and what form it should have. This ensures that names and values of attributes remain consistent across all implementations of all telemetry data [13].

2.4.3 APIs and SDKs

OpenTelemetry provides an API and an SDK for the most popular programming languages. The OpenTelemetry API contains the interfaces required to instrument code. The API itself is completely decoupled from the actual generation of telemetry data; that is what the SDK does. Instead, the API comes with a "no-op" SDK implementation that implements the API interfaces but does nothing [10, 13].

The OpenTelemetry SDK implements the actual functionality of the API. It is responsible for things like generating, aggregating, and transmitting the telemetry data [13]. As the SDK and API are decoupled it is possible to create custom SDKs to fit specific needs. OpenTelemetry also has plugins called "exporters" external to the SDK whose job is to translate and transmit telemetry data to backends in certain formats [10], OpenTelemetry has their own format called OTLP. Figure 8 illustrates how these components are connected.

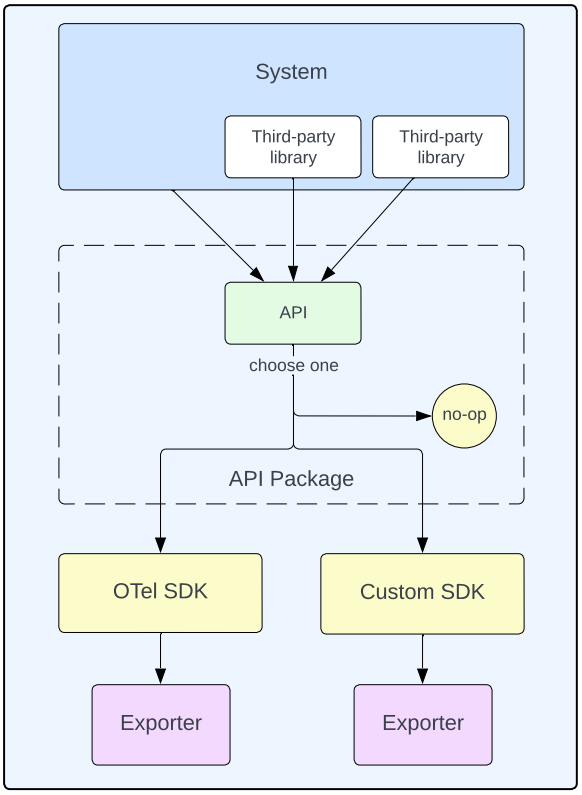


Figure 8: A system utilizing the OpenTelemetry API, based on Figure 3.1 in [10].

Someone developing a library that is not meant for use on its own would utilize the OpenTelemetry API with the no-op SDK that comes with it. As long as an SDK is

not used, the library will not emit telemetry, which means the users are free to choose whether or not to utilize the instrumentation. Developers of a system that might depend on libraries would utilize the API *and* the SDK. This enables the already instrumented libraries to emit telemetry and also enables the system code itself to be instrumented. However, it might be inconvenient or impossible to add these dependencies or change the source code in some systems. Automatic instrumentation is a powerful alternative in those cases.

2.4.4 Automatic Instrumentation

OpenTelemetry offers substantial support for automatic instrumentation, also commonly referred to as *auto-instrumentation* or *zero-code instrumentation*. The purpose of automatic instrumentation is to reduce the investment needed to get value from OpenTelemetry by making it possible to add observability without having to modify any source code.

A system that utilizes automatic instrumentation will generate telemetry through the libraries it depends on, either natively instrumented or libraries with instrumentation libraries. Natively instrumented libraries have implemented the OpenTelemetry API and will emit telemetry once paired with an SDK [28]. OpenTelemetry provides helper libraries that enable observability for some popular libraries that are not natively instrumented, these helper libraries are known as "instrumentation libraries" [22]. The technical implementation of instrumentation libraries varies from language to language. In Java, bytecode injection is utilized [13] which revolves around modifying compiled `.class` files at runtime [29].

Automatic instrumentation can be seen as a bundle consisting of the API, SDK, and instrumentation libraries. Libraries that have implemented the OpenTelemetry API emit telemetry through the SDK, and libraries that have not implemented the API will emit telemetry through their instrumentation library if they have one.

An evident drawback to automatic instrumentation is that no system-specific telemetry will be generated. With automatic instrumentation on its own, it is not possible to generate telemetry in a piece of code that does not use any external libraries. What the actual telemetry looks like is also largely beyond the control of automatic instrumentation users. Another drawback is that it cannot be assumed that every library is natively instrumented or has an available instrumentation library.

3 Related Work

The importance of minimizing performance overhead within observability tools has been pointed out in the past [14, 30]. The researchers behind Dapper made minimizing performance overhead a key design goal, highlighting that performance degradation would be a major obstacle to the adoption rate of their tracing tool [14]. Similar conclusions were drawn by the researchers behind the tracing tool JCallGraph who argue that end users would simply turn their tool off if it had a major impact on performance [30]. Five bodies of work have been identified that relate to the topic of performance overhead in

OpenTelemetry [31, 32, 33, 34, 35].

A custom tail-based sampling framework for OpenTelemetry is created and compared to the standard built-in OpenTelemetry implementation in [31]. The memory overhead was observed to be approximately a third lower for the custom-built tail-based sampling framework. This work also examined how tracing with OpenTelemetry impacted latency for a specific service, finding an increase of between 9% and 16%.

DoorDash Engineering has a detailed blog post about the resource usage of the Java Batch Span Processor (BSP) in OpenTelemetry [32]. Certain configurations for the built-in BSP were tested but deemed to have no impact on relieving CPU overhead. The blog post goes on to investigate different approaches to customizing the BSP and evaluates how performance is impacted in terms of throughput (operations per second) and CPU time. They concluded that utilizing an MPSCQueue (Multiple Producers, Single Consumer Queue) with signal batching had the best performance.

A comparison of performance overhead caused by implementing tracing with OpenTelemetry, Kieker, and inspectIT is made in [33], and overhead is evaluated in terms of method execution time. The results show that Kieker performed the best, with OpenTelemetry in second place and inspectIT in last place.

OpenTelemetry automatically publishes certain benchmarks to track and showcase changes in performance across different updates [34]. The collector component has a very thorough set of benchmarks that track CPU and memory performance under various conditions. For example, the average CPU usage when idle has been under 0.5% for most updates. For language SDKs only JavaScript and Java benchmarks are currently being published automatically. The SDK benchmarks measure how fast certain operations are. For instance, the JavaScript SDK can create a span with 10 attributes almost 700 000 times per second under the benchmarking conditions.

Automatically instrumented tracing with OpenTelemetry is implemented in a legacy application in [35], the paper examines how much additional latency this causes. The mean latency was observed to be between 1.86% and 2.56% higher, depending on load. The 99th percentile latency was observed to be between 2% and 4% higher with OpenTelemetry, again dependent on load.

The mentioned related work has either focused on specific components of OpenTelemetry [31, 32, 34] or more broadly on OpenTelemetry's system-wide impact but for specific types of overhead [33, 35]. This thesis aims to fill a gap in the literature by examining the performance implications of integrating OpenTelemetry into a system while considering all relevant forms of overhead. Furthermore, all the mentioned previous work has been focused entirely on OpenTelemetry for tracing, and this thesis also aims to examine overhead for metrics.

4 Methodology

This section describes the experimental environment and how it was used to produce the results presented in this thesis.

4.1 Experimental Environment

The environment used to evaluate performance overhead consists of multiple parts: a microservice-based system, a load generator, a performance observer, and two machines on which the different components run.

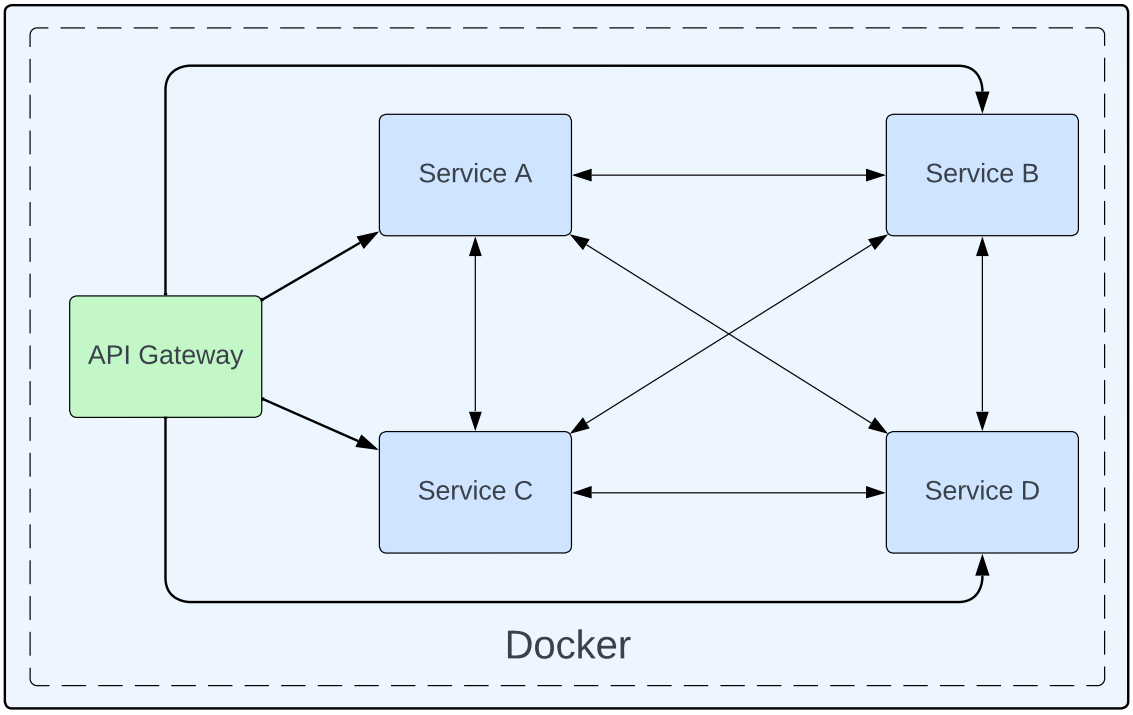


Figure 9: The system used to evaluate performance overhead.

The system on which performance is measured implements a microservice architecture using the API gateway pattern mentioned in Section 2.1. The system consists of five services, their relationship is visualized in Figure 9. The API Gateway is responsible for receiving requests from outside the system and routing them to one of the four other services, the sender of the request decides which one. When a service receives a request from the gateway it will sequentially send 60 requests split between the other non-gateway services, alternating between them. A non-gateway service that receives a request from another non-gateway service will immediately return. All services are implemented in Spring[1], communicate over REST and are deployed as containers in Docker[2].

The load generator is responsible for sending requests to the API Gateway. It is implemented in the Python-based load testing tool Locust[3]. It simulates a configurable, static amount of users that send a request, receive the response, wait between 1–1.1 seconds, send a request, and so on. A user sends each request to a random service but must have sent an equal number of requests to all services before repeating a request; this ensures an even load between services. The latency for every request is measured and recorded by the load generator.

[1] `https://spring.io/`
[2] `https://docker.com/`
[3] `https://locust.io/`

The performance observer is a simple shell script that pipes the output from the command `docker stats` to a text file in a format that can be parsed by plotting tools. For every container, roughly once per second, the performance observer pulls metrics such as current CPU usage, current memory usage, and total network usage.

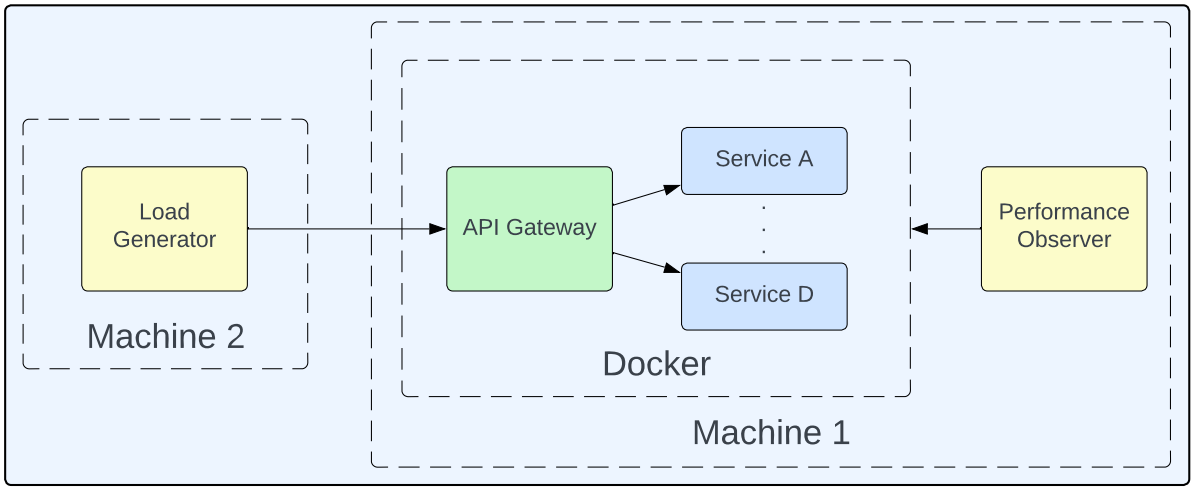


Figure 10: The experimental environment.

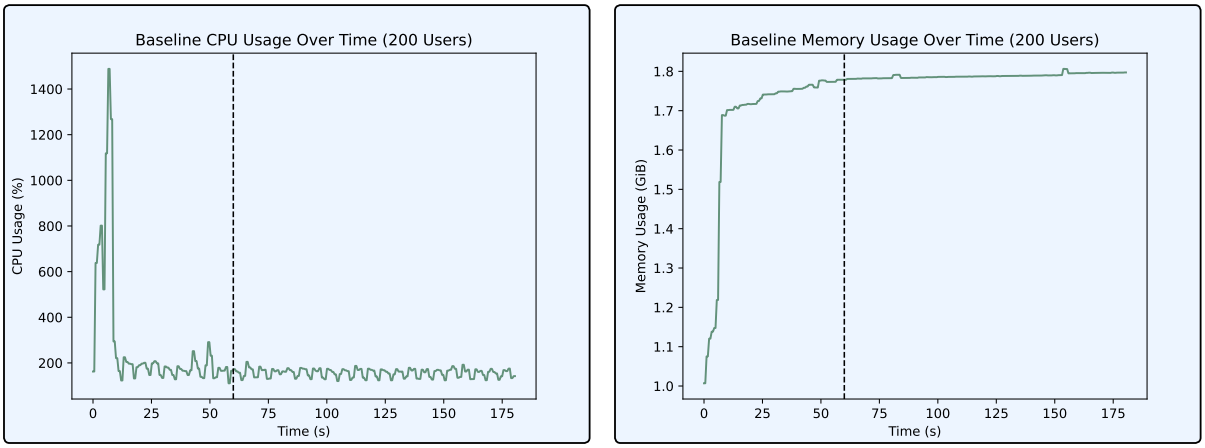
The components of the environment are running on two machines as illustrated in Figure 10. Machine 1 uses an AMD Ryzen™ 7 5800X CPU with 16 logical processors and 32GiB of 3600MHz RAM. Since the system has 16 logical cores available the maximum CPU usage at any time is 1600%. Machine 2 uses an Intel® Core™ i7-8565U CPU with 8 logical processors and 16GiB of 2400MHz RAM. There is no meaningful latency between the machines.

4.2 Methodology

All evaluated tools and components are deployed as part of the system's containers or as individual containers. This allows for the performance observer to capture the performance impact of any added tools.

The following process was used for all tests:

1. Start the system on Machine 1, with any implemented tools.
2. Wait until the system has initialized fully.
3. Start the load generator on Machine 2.
4. Start the performance observer on Machine 1.
5. After 3 minutes, stop all components.
6. Remove the values observed during the first 60 seconds from the data.



(a) Typical CPU usage behavior.

(b) Typical memory usage behavior.

Figure 11: Example of a warm-up period, all data to the left of the dashed line is discarded before analyzing the results.

The purpose of step 6 is to let the system warm up. This is necessary as the system's performance was observed to be volatile when initially put under load. This behavior could, for example, stem from caches not yet being populated or the system not being fully compiled due to Just-In-Time compilation. Figure 11 a) and b) visualize this volatility, only the data to the right of the dotted line is analyzed.

4.3 Implementation of Tools

Seven different variations of the system exist, each with a different set of observability tools. The following subsections describe the different variations and provide details of what has been implemented. Unless otherwise specified, all tools use the default configurations.

4.3.1 Baseline

The baseline is the basic version of the system, as described in Section 4.1. Sections 4.3.2–4.3.7 describe system variations that can be seen as the baseline with certain tools implemented.

4.3.2 OTel Manual Tracing

This version of the system is manually instrumented to emit traces with OpenTelemetry. To achieve this, the OpenTelemetry API and SDK are manually added as dependencies to all services in the system. The SDK is configured using the automatic configuration module, which enables setting up the SDK through environment variables instead of doing it programmatically. The source code is modified to create spans and to inject and extract the context to and from the HTTP header sent with requests between services. Every request generates one trace with 123 spans, each span has 8 tags.

Version 1.36.0 of the OpenTelemetry bill of materials is used. This specifies the versions of all other OpenTelemetry components. The OpenTelemetry Protocol (OTLP) exporter is used to export spans over gRPC from the services to Jaeger[4], a popular open-source tracing observability backend. Exported spans are compressed with gzip. Jaeger is configured to store the 50 most recent traces in memory, deleting the oldest traces as new ones come in. Version 1.56 of Jaeger is used.

4.3.3 OTel Auto Tracing

In this version of the system, the OpenTelemetry Java agent is used to instrument the system to emit traces automatically. This is achieved by adding the Java agent JAR to the JVM startup arguments of all services in the system. No changes to the source code are necessary for this to work. Each request generates a trace with 123 spans, each span has 36 tags. It appears that the only way to regulate the number of tags in a span that is created automatically is to modify it after it has already been created. This is inconvenient and requires additional computational effort. Therefore, the number of spans is left as is.

Version 2.2.0 of the agent is used. The logs and metrics exporters are set to *none*. Spans are exported with gzip compression to Jaeger over HTTP using the OTLP exporter. Jaeger is otherwise configured exactly like in *OTel Manual Tracing*.

4.3.4 Zipkin/Zipkin

Zipkin[5] is a popular open-source tracing tool that has instrumentation as well as an observability backend component. This version utilizes both of these components.

Zipkin instrumentation is implemented with micrometer[6], which acts as a facade that instruments the system through Brave, a Zipkin-based tracing bridge. Every request generates 123 spans and every span contains 10 tags. Version 3.1.1 of Zipkin is used for the observability backend component, it is configured to store the 50 most recent traces in memory.

4.3.5 Zipkin/Jaeger

This version of the system is instrumented with Zipkin in the same way as *Zipkin/Zipkin* is, but with Jaeger as the observability backend component. Jaeger is set up the same way as in *OTel Manual Tracing* apart from the fact that it accepts spans on Zipkin's format.

[4] `https://jaegertracing.io/`
[5] `https://zipkin.io/`
[6] `https://micrometer.io/`

4.3.6 OTel Auto Metrics

The OpenTelemetry Java agent is used to expose metrics automatically in this system version. It is deployed largely in the same way as in *OTel Auto Tracing*, the logs and traces exporters are set to *none*, and trace generation is suppressed by sampling 100% of the traces.

The OpenTelemetry agent makes each service expose 49 different metrics, most of them related to the JVM the service is running on. Two examples are the total number of CPU seconds used and the peak number of threads used.

The popular open-source metric tool Prometheus[7] version 2.51.0 is used as the observability backend. The OpenTelemetry Prometheus exporter is used to expose metrics in the Prometheus data format, the Prometheus backend component scrapes these metrics once every 15 seconds.

4.3.7 Prometheus Metrics

Prometheus is used both as an observability backend and to instrument the services in this version of the system. Micrometer is used as a facade that uses Prometheus to gather metrics from the services. This automatically exposes all the metrics the OpenTelemetry Java agent did and 48 additional metrics meaning a total of 97 metrics are available. As an observability backend, Prometheus is configured in the same way as in *OTel Auto Metrics*.

5 Evaluation

The results of the experiments are presented in this section. The individual experiments were replicated seven times. The distribution of average CPU and memory usage as well as the 95th percentile latency are extracted from these seven tests. There were no meaningful difference in how many requests were handled between tests with the same load.

5.1 Tools and Deployments

The results of using different tools and deployment methods mentioned in Section 4.3 are presented here. The load generator was configured to emulate 200 users, which equates to roughly 150 requests per second. The results from all individual tests are presented in the appendix; see Figures A1–A3. The average CPU usage, average memory usage, and 95th percentile latency from each test are presented in Figures 12–14.

[7] `https://prometheus.io/`

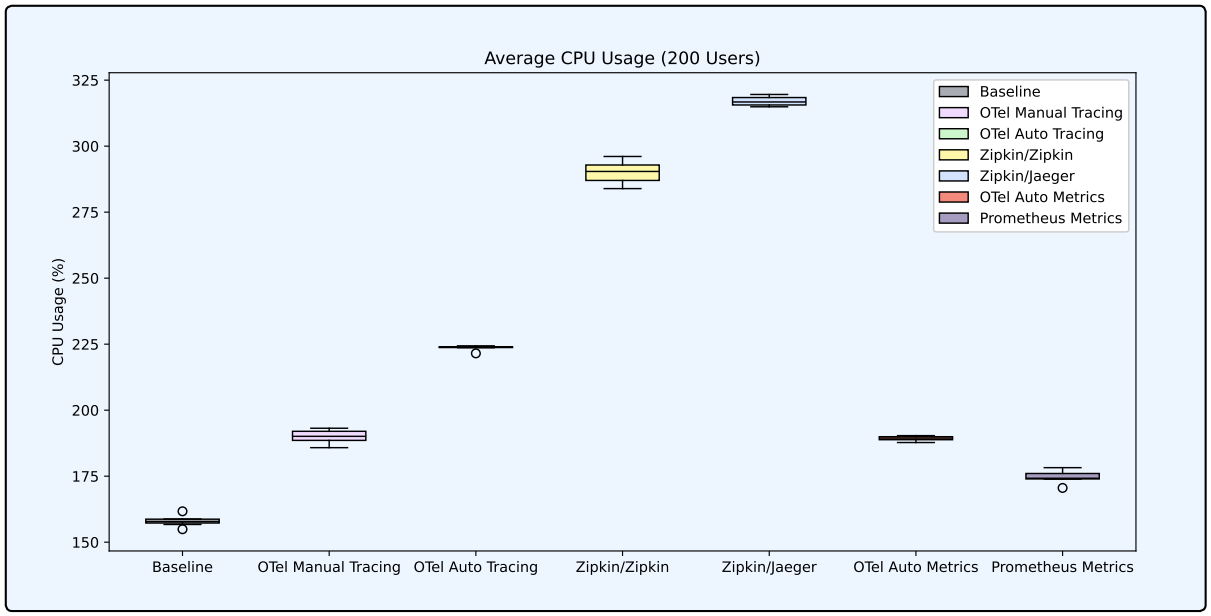


Figure 12: Distribution of average CPU usage, categorized by the various tools and deployment methods used.

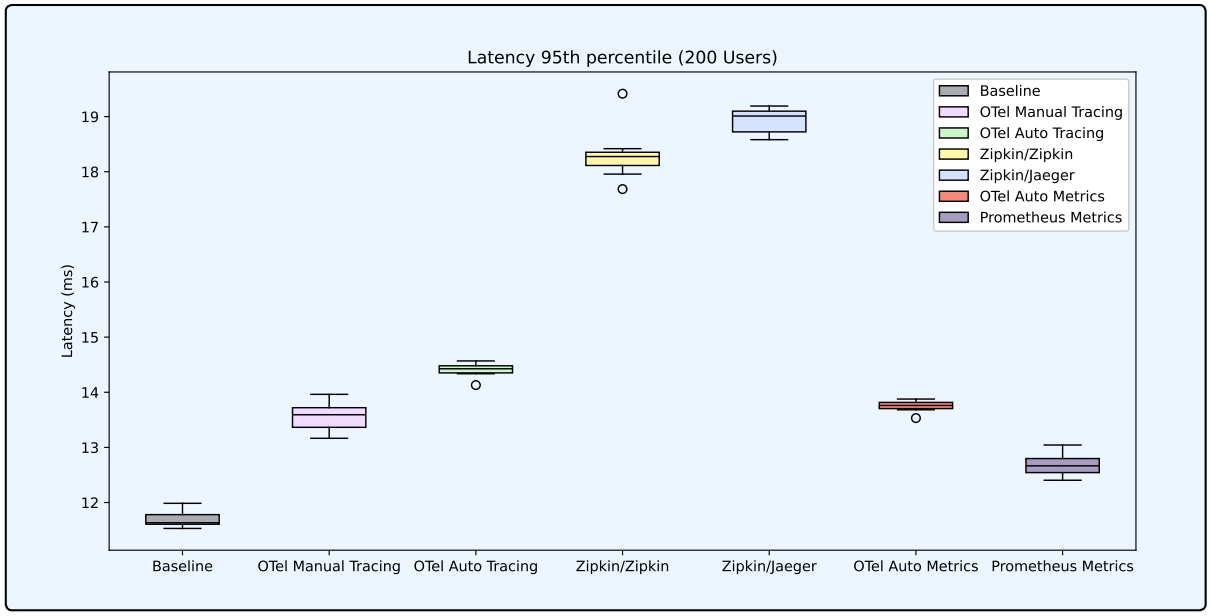


Figure 13: Distribution of 95th percentile latency, categorized by the various tools and deployment methods used.

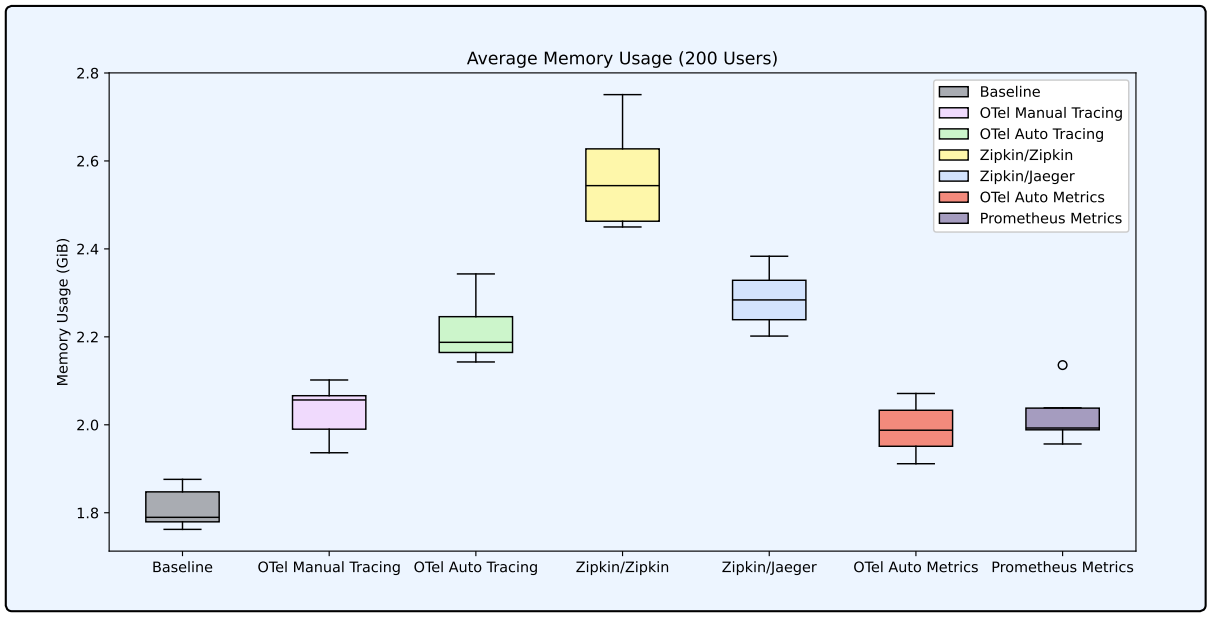


Figure 14: Distribution of average memory usage, categorized by the various tools and deployment methods used.

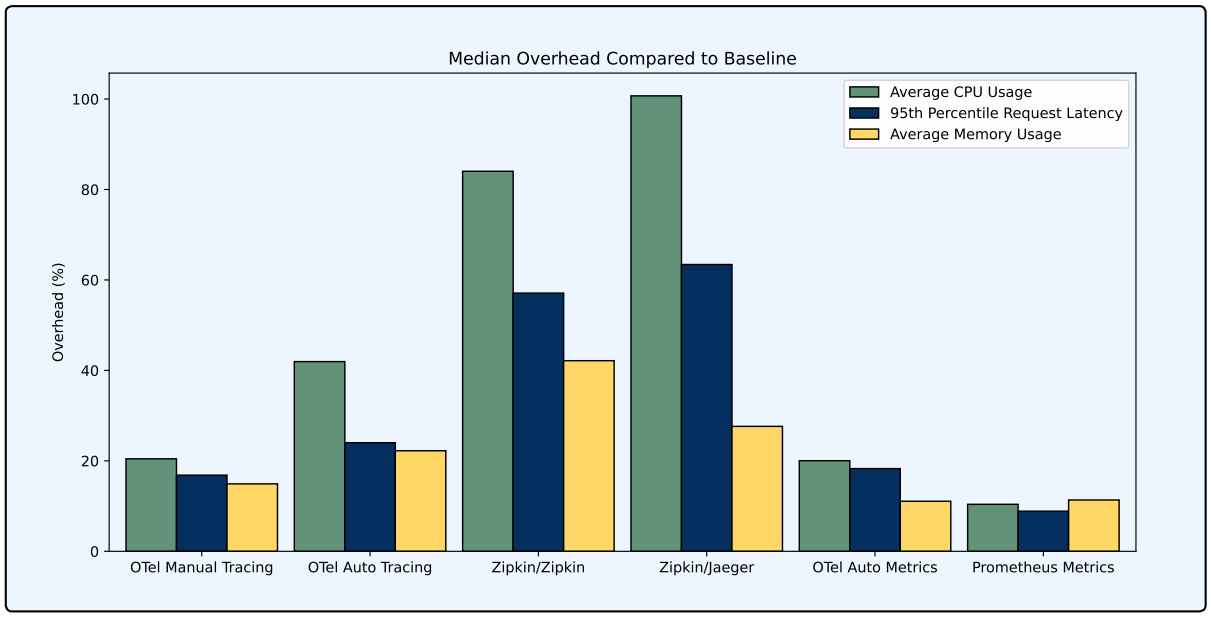


Figure 15: Median overhead of using various tools and deployment methods compared to the baseline.

The first thing to note from the results in Figures 12–14 is that the baseline performed the best in all cases. This is consistent with the basic premise of this thesis: observability tools add overhead. Figure 15 presents the median percentage overhead of the other system versions compared to the baseline.

The two variations of the system that utilized Zipkin for trace instrumentation had a larger overhead compared to its OpenTelemetry counterparts in terms of CPU usage, request latency, and memory usage. As a backend, Zipkin performed better than Jaeger

in terms of CPU usage and latency while Jaeger performed better in terms of memory usage.

For all performance types, adding observability in the form of metrics resulted in less overhead compared to adding tracing to the system. Using Prometheus to instrument the system caused less CPU usage and latency overhead compared to using OpenTelemetry's Java agent for metrics. OpenTelemetry's agent performed slightly better in terms of memory usage, but the difference was minimal.

Manual tracing with OpenTelemetry caused less overhead than automatic tracing. There are multiple reasons why this might be. For example, it is reasonable to expect the instrumentation libraries used when automatically tracing to be less efficient than using the OpenTelemetry SDK directly. The automatic instrumentation also created spans containing more tags than the manually instrumented traces.

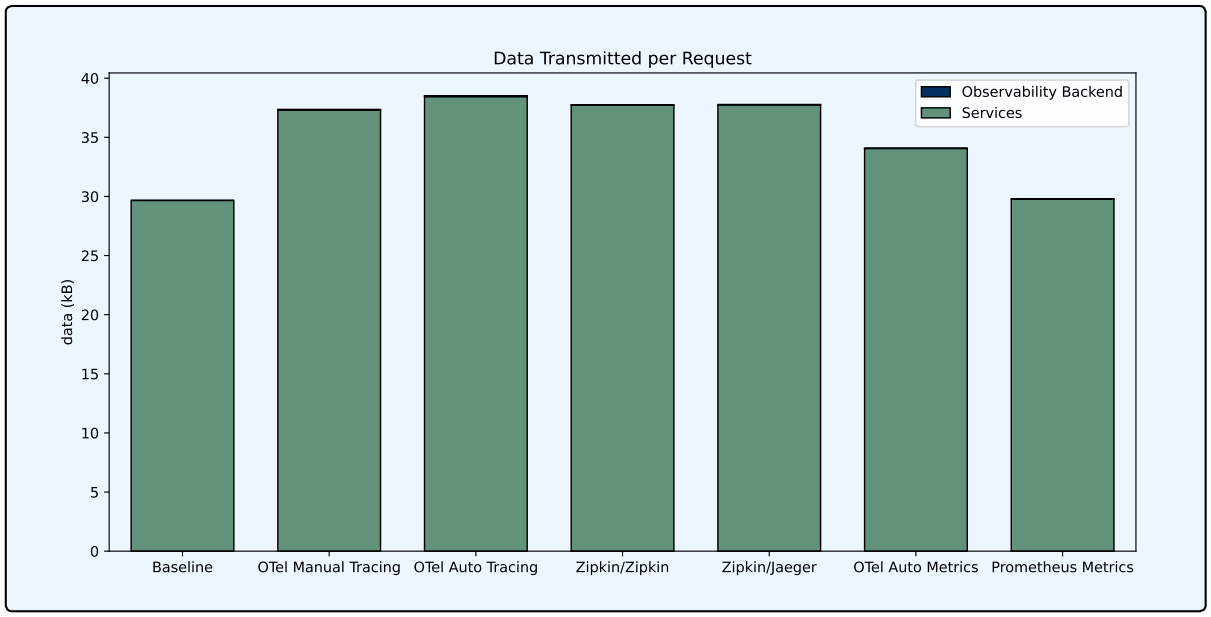


Figure 16: Data transmitted over the network per request, categorized by the various tools and deployment methods used.

Figures 16 and 17 show how much data was transmitted and received over the network per request by the microservices in the system. Each bar represents the average from the seven tests. There was practically no variance in network usage between tests of the same system variation. Most network traffic happens between the containers in the system, which is reflected in the fact that the total amount of network transmitted and received is almost identical for each system variation.

The dashed lines in Figure 17 mark the top of the baseline, this is the amount of network needed to send requests between services. The green part of the bar that is above the dashed line can be attributed to *context* being passed between services together with requests (see Figure 6). The amount of network overhead that comes from context propagation is roughly the same between the versions of the system that have tracing.

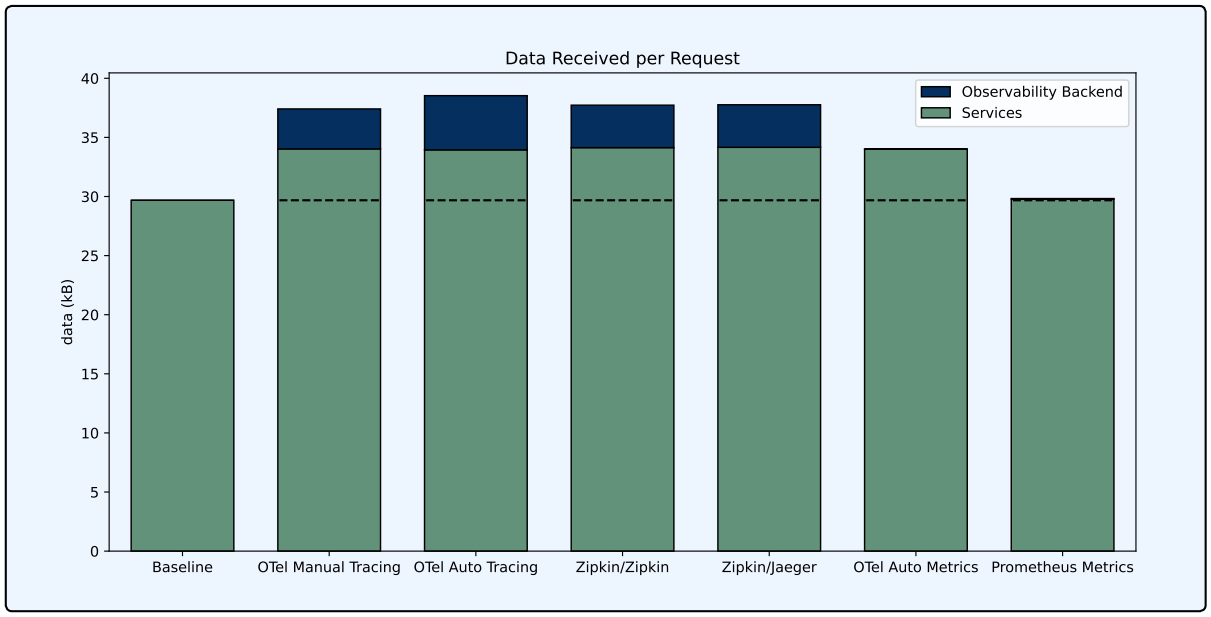


Figure 17: Data received over the network per request, categorized by the various tools and deployment methods used.

It seems like the version of the system that generates metrics through OpenTelemetry also uses network for context propagation, even though no traces are generated. Typically metrics do not require context to be propagated. For example, the Prometheus instrumented system generates the same metrics (and more) without any context propagation. This happens because of how traces were turned off in the agent. Even though no traces are sampled or exported, context is still propagated; the cause of this is discussed more in-depth in Section 5.3. This might also contribute to the difference in CPU usage and latency between the metric-generating versions of the system. No way of fully turning off tracing within the OpenTelemetry Java agent was found. This makes it less appealing to use it solely for metric instrumentation. However, if the agent is used for both metrics and traces this would not be an issue.

The blue part of the bars in Figure 16 are barely visible which means that not much data is transmitted by the observability backends. This is reasonable as the main purpose of an observability backend is to collect and store data. In Figure 17 it is apparent that the observability backends are receiving sizable amounts of data in the versions of the system that have tracing implemented. The versions that generate metrics both show that Prometheus is not receiving any significant amount of data. This is somewhat expected as metrics are configured to be scraped relatively infrequently. Also, the metrics themselves do not require much data to be represented as most of them are integers.

The Zipkin system versions have very similar network results. This is expected since the only difference between them is the observability backend they use, which should not impact network usage in most cases. In the OpenTelemetry tracing versions, there is some difference in how much data the observability backends receive. Most of this can likely be attributed to the fact that the spans being generated by the Java agent contain more information. The traces generated by the manually traced OpenTelemetry version contain roughly as much data as those generated by the Zipkin versions. Their observability backends receive essentially the same amount of data. This suggests that

there is no large difference in network overhead between OpenTelemetry and Zipkin when transporting spans.

5.2 Batching

OpenTelemetry uses batching by default. There is also support for making various adjustments. One of the most impactful adjustments is modifying the maximum allowed batch size, which dictates the maximum amount of spans a batch may contain. In OpenTelemetry this value is by default set to 512. To explore the performance implications of adjusting this value, it was modified in the system version which is manually instrumented to emit traces with OpenTelemetry. The load generator was configured to emulate 400 users, resulting in roughly 300 requests being sent per second. The results from the individual experiments are presented in Figures A11–A13, with Figures A14–A16 presenting the average CPU usage, average memory usage, and 95th percentile latency.

Figure 18 presents the median overhead for average CPU usage, average memory usage, and 95th percentile latency. As the maximum batch size is decreased, the CPU overhead increases steadily from 18.4% to 49.0%. Most of the difference comes from the services themselves and not Jaeger, the observability backend, indicating that exporting batches is CPU-intensive. The latency overhead also increases but at a slower pace and seems to plateau at around 23%.

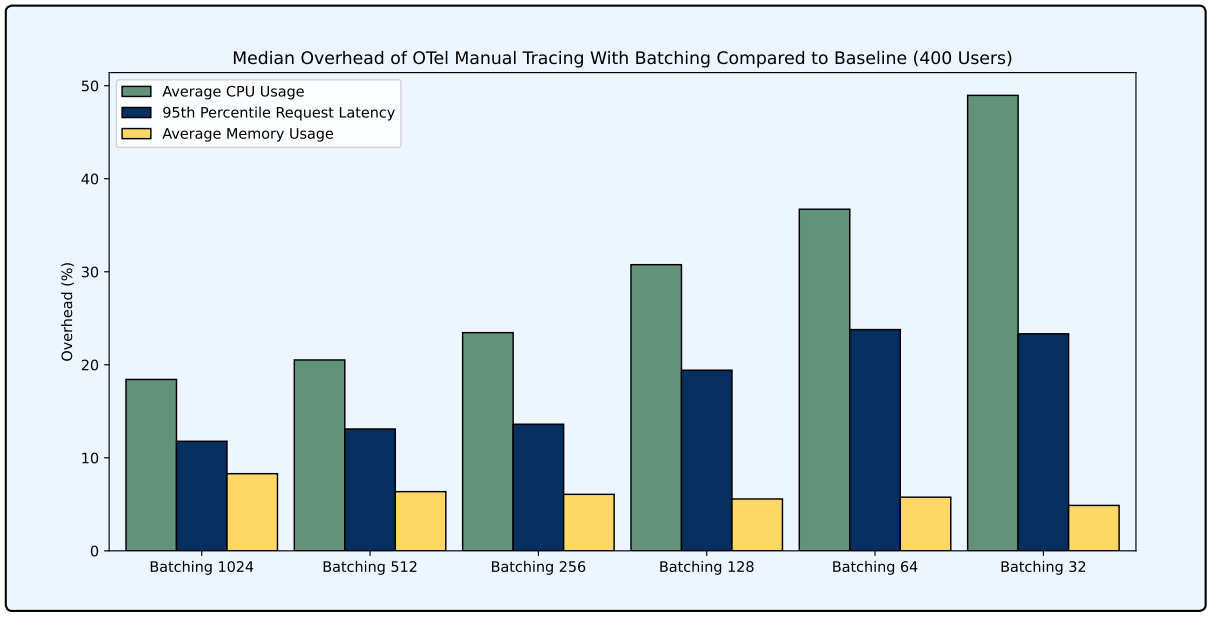


Figure 18: Median overhead of OTel Manual Tracing with various maximum allowed batch sizes compared to the baseline.

As the maximum batch size decreases, the median overhead for average memory usage slightly decreases from 8.3% to 4.9%. This observation aligns with the theory of batching since smaller batch sizes should result in fewer spans waiting to be exported in the buffer. However, the decrease in memory usage is relatively small compared to the increase in

CPU and latency overhead. This indicates that using a larger batch size is an efficient way of exchanging memory for CPU and latency.

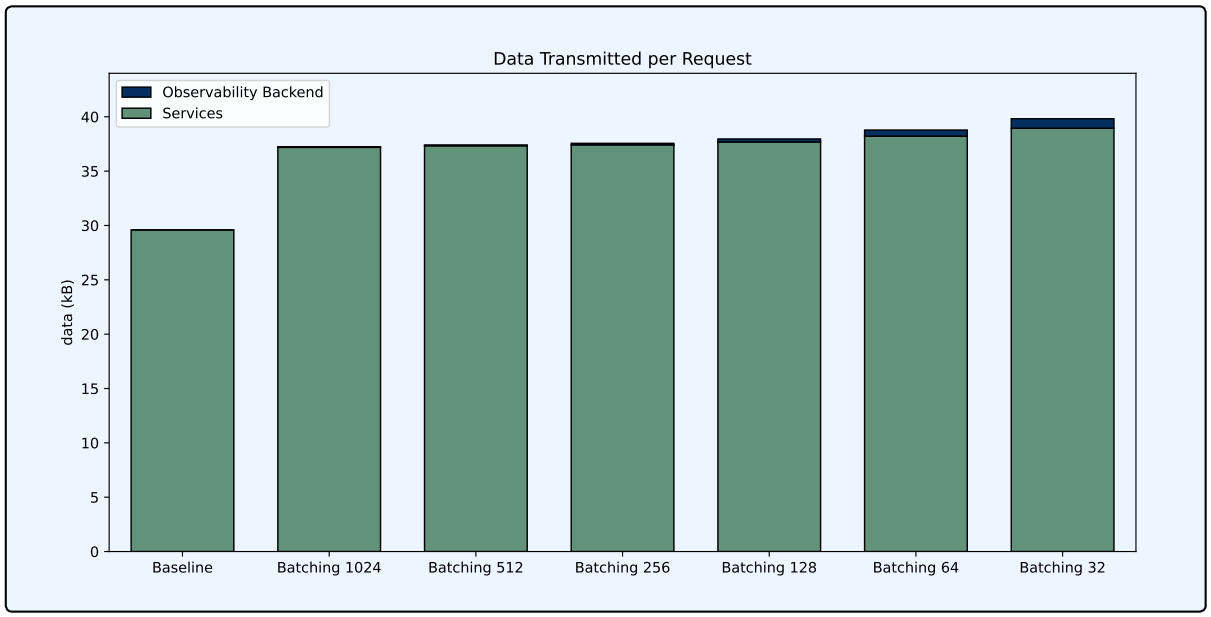


Figure 19: Data transmitted over the network per request, categorized by the baseline and OTel Manual Tracing with various sample sizes.

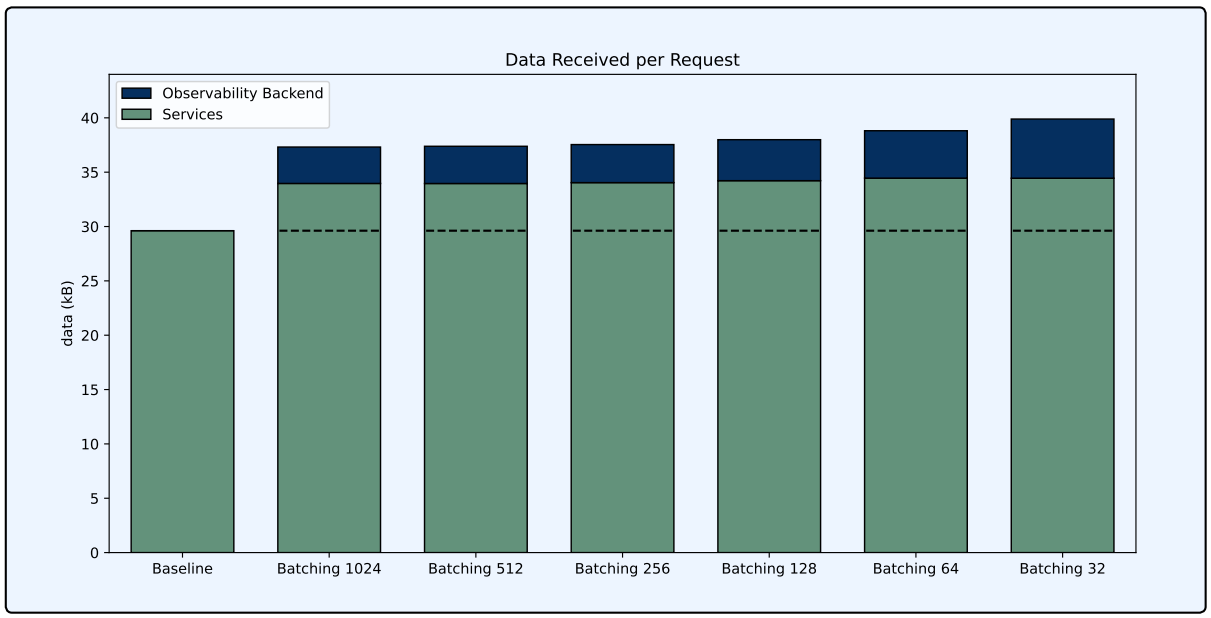


Figure 20: Data received over the network per request, categorized by the baseline and OTel Manual Tracing with various sample sizes.

When examining the data transmitted over the network in Figure 19, an interesting observation can be made: Jaeger transmits an increasing amount of data when the maximum

batch size decreases. Although Jaeger does poll the microservices for purposes like adaptive sampling [18], this behavior is somewhat unexpected as none of these features are utilized. No other explanation or way to turn this off was found. The results observed in Figure 20 are more expected, showcasing an increase in the amount of data Jaeger receives as batch sizes decrease. The difference is likely the protocol overhead mentioned in Section 2.3.1.

5.3 Sampling

Head-based sampling was implemented in the system version which utilized OpenTelemetry to manually instrument traces, the default batch settings were used. OpenTelemetry's `parentbased_traceidratio` was used with various sampling probabilities to investigate the effectiveness of sampling to reduce overhead. The load generator was configured to emulate 300 users resulting in roughly 225 requests sent per second. The results from the individual experiments are presented in Figures A4–A6, the average CPU usage, average memory usage, and 95th percentile latency are presented in Figures A7–A9.

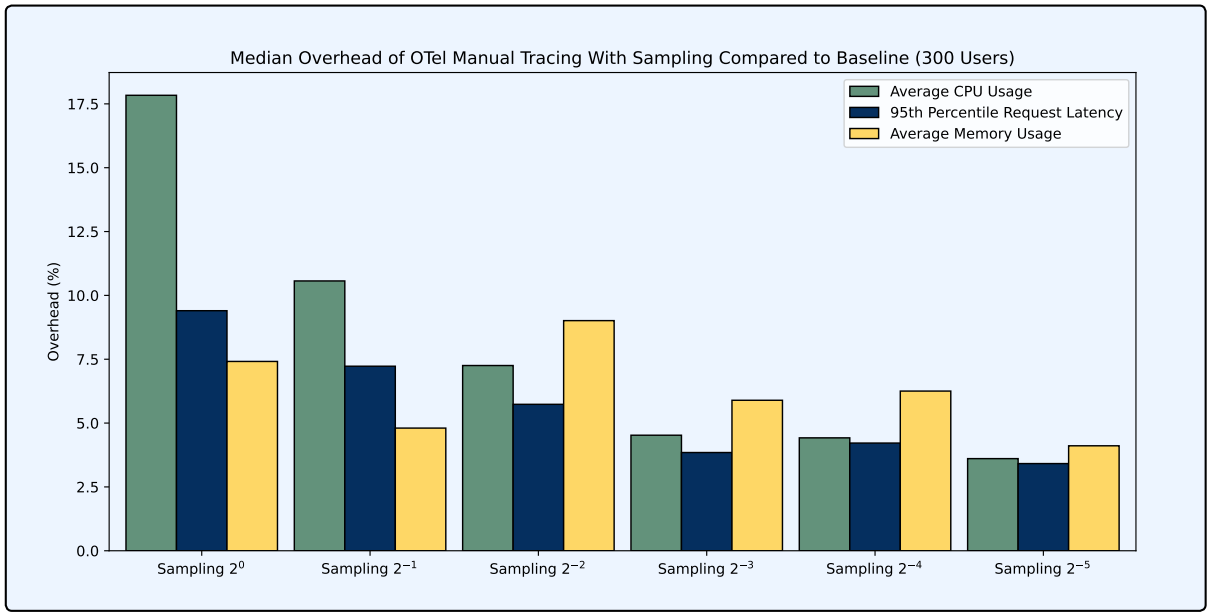


Figure 21: Median overhead of OTel Manual Tracing with various sampling probabilities compared to the baseline.

The median overhead compared to the baseline is presented in Figure 21. It is clear that sampling fewer traces decreases the CPU usage of the system. The median overhead in average CPU usage steadily decreases from 17.8% to 3.6% as fewer traces are sampled. The median overhead in 95th percentile latency similarly decreases from 9.4% to 3.4%. No such trend can be observed when looking at memory usage, suggesting sampling is ineffective at reducing memory overhead.

The data transmitted and received over the network can be seen in Figures A10 and 22 respectively. The amount of data received by the observability backend closely mirrors the amount of traces sampled. This is expected as traces that are not sampled, by definition,

should not be sent to the observability backend. The amount of data sent and received for the purpose of propagating context remains consistent across all sampling probabilities. This is consistent with what was observed in Figure 17.

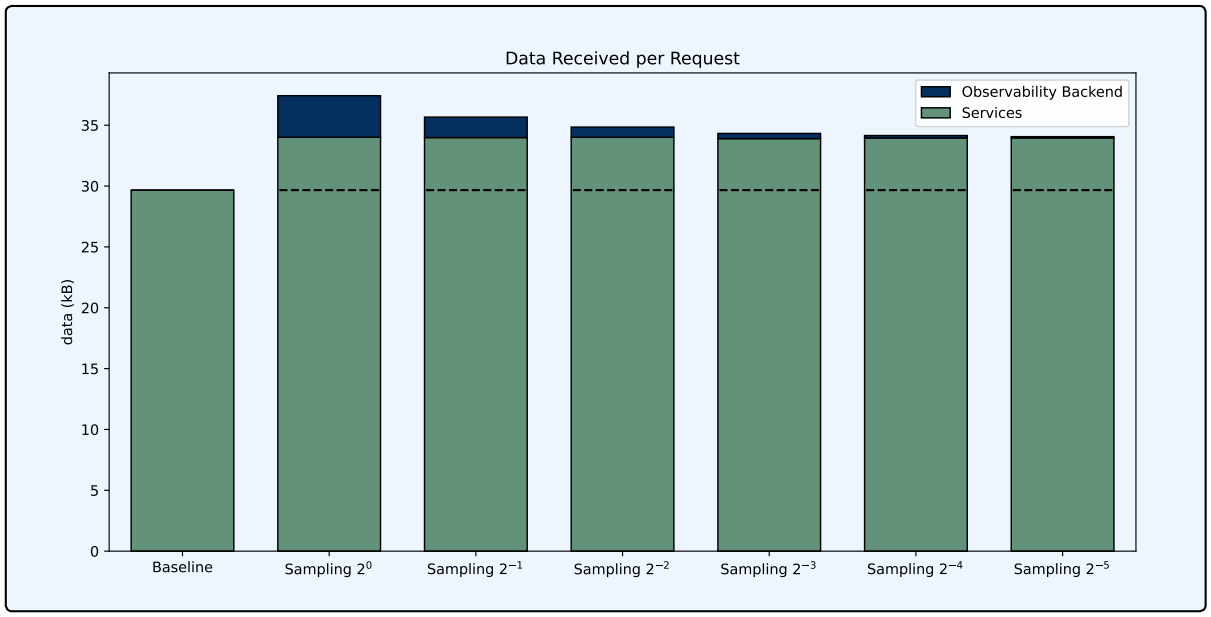


Figure 22: Data received over the network per request, categorized by the baseline and OTel Manual Tracing with various sample sizes.

In OpenTelemetry, context has to be propagated even when traces are not sampled, as the context contains the sampling decision. However, it is reasonable to assume that most of the information in the context could be removed if the trace is not sampled. For instance, if the trace is not sampled, the next span in the trace does not need to know its parent ID. This unexpected behavior likely stems from the fact that the upstream sampling decision can be ignored downstream. For this to be possible, no information can be removed from the context.

6 Summary and Conclusion

This thesis has discussed the challenges of debugging systems that utilize microservice architectures and how observability tools like OpenTelemetry can be used to tackle these challenges, at the cost of performance. The research questions asked at the beginning of the thesis were answered by creating a microservice-based system in which observability tools were implemented. These questions can now be revisited.

1. **How is performance impacted by implementing OpenTelemetry for traces/-metrics?**

The experiments clearly demonstrate that using OpenTelemetry decreased the system's overall performance. For CPU, memory, latency, and network respectively, overheads

as large as 42%, 22%, 24%, and 30% were recorded. Adding observability with OpenTelemetry in the form of metrics had less impact on performance than adding traces. Additionally, manually instrumenting the system to emit traces had a lower impact on performance than doing so automatically with the Java agent.

2. **How does the above compare to other prevalent tools?**

In terms of tracing, OpenTelemetry outperformed Zipkin in CPU usage, memory usage, and request latency. There was no significant difference in network overhead. When it comes to instrumenting metrics, Prometheus proved to be less taxing than the OpenTelemetry Java agent in terms of CPU usage, network usage, and request latency. It is worth noting that the comparison between the two might not be entirely comparable, as the OpenTelemetry Java agent does not seem to be designed solely for metric instrumentation.

3. **How effective are strategies such as batching or sampling at reducing the performance impact of OpenTelemetry for tracing?**

Batching has been identified as an effective way to reduce the impact on CPU usage, network traffic, and request latency that may result from OpenTelemetry. Although larger batch sizes were observed to increase memory consumption, the increase was relatively small, especially when compared to the decrease in CPU usage and latency. This suggests that batching is an efficient trade-off and that it makes sense for OpenTelemetry to use batching by default.

Using head-based sampling together with batching reduced the CPU, latency, and network overhead even further. With the default maximum batch size and a sampling rate of approximately 3%, the network overhead was roughly 4 kB, while the CPU and latency overhead was almost negligible at around 3.6% and 3.4%, respectively. This indicates that head-based sampling is an effective way of reducing overhead and that OpenTelemetry can be configured with very low levels of overhead.

6.1 Limitations

The method employed to evaluate overhead in this thesis has certain limitations. Firstly, only one test system was utilized for this thesis. As this system was implemented in Java, only Java components were evaluated. Other programming languages have their own SDKs and agents that might perform differently. Furthermore, it is possible that different architectures or hardware constraints may produce different results.

Some simplifications were made to make it easier to build the test system. Most notably, all observability data is stored in memory by all observability backends. This way of storing data is not persistent and expensive; a production-grade system would likely utilize a database instead. Unfortunately, different observability backends integrate with different databases, and configuring and setting all of these up was considered out of scope due to the time constraints. There were also concerns that different databases might have different impacts on performance which would make comparisons too unrealistic.

6.2 Future Work

With the previously mentioned limitations in mind, there are several ways to expand upon the work presented in this thesis. Below are some suggestions.

Evaluating the Consistency of OpenTelemetry's Overhead Across Different Microservice Patterns
As previously mentioned, one of the limitations of this thesis is that OpenTelemetry's performance was only evaluated for one microservice system. It would be interesting to examine whether the findings of this thesis hold true across different microservice patterns. One interesting tool for this purpose is µBench [36] which allows the creation of various mock microservice systems with custom patterns for the purpose of benchmarking.

Pinpointing the Sources of Overhead Within OpenTelemetry
In the evaluation section of this thesis, a lot of reasoning was used to explain performance differences. Tools like profilers could provide a deeper understanding of the reason behind differences in performance overhead. For example, the cause behind manual trace instrumentation performing better than automatic instrumentation in OpenTelemetry could be investigated further.

References

[1] K. Gilliusson, *Evaluation of Distributed Tracing Frameworks for Microservice Observability*. 2020.

[2] C. Richardson, *Microservices patterns: With examples in Java*, pp. 8–14, 259–263. Manning, 2018.

[3] Y. Shkuro, *Mastering Distributed Tracing: Analyzing performance in microservices and complex systems*, pp. 8–10, 66. Packt Publishing Ltd, 2019.

[4] A. Singleton, "The economics of microservices," *IEEE Cloud Computing*, vol. 3, no. 5, pp. 16–20, 2016.

[5] S. Gadge and V. Kotwani, "Microservice architecture: Api gateway considerations," *GlobalLogic Organisations*, vol. 11, 2017.

[6] B. Li *et al.*, "Enjoy your observability: An industrial survey of microservice tracing and analysis," *Empirical Software Engineering*, vol. 27, p. 2, 2021.

[7] R. E. Kálmán *On the General Theory of Control Systems*, August 1960.

[8] G. M. Charity Majors, Liz Fong-Jones, *Observability Engineering: Achieving Production Excellence*, pp. 4–6. 55–57, 63–65, 74–75. O'Reilly Media, 2022.

[9] CNCF Technical Advisory Group for Observability, *Observability Whitepaper v 1.0*. October 2023. [online], Available at: `https://github.com/cncf/tag-observability/blob/whitepaper-v1.0.0/whitepaper.md`, Accessed: 2024-01-24.

[10] A. Parker *et al.*, *Distributed Tracing in practice*, pp. 14–16, 34–37, 136–138, 226–227. O'Reilly Media, 2020.

[11] M. Usman, S. Ferlin, A. Brunstrom, and J. Taheri, "A survey on observability of distributed edge container-based microservices," *IEEE Access*, vol. 10, p. 86913, 2022.

[12] N. Harjunpää, "Log management system technologies and methods for near real-time fault analysis systems: An exploration of log shipping and storage," pp. 10–12, 17–18, 46–47, 2023.

[13] A. Boten, *Cloud native observability with OpenTelemetry*, pp. 9, 15–24, 36–38, 60–71, 331–337. Packt Publishing Limited, 2022.

[14] B. H. Sigelman, L. A. Barroso, M. Burrows, P. Stephenson, M. Plakal, D. Beaver, S. Jaspan, and C. Shanbhag, "Dapper, a large-scale distributed systems tracing infrastructure," tech. rep., Google, Inc., 2010.

[15] OpenTracing, *The OpenTracing Semantic Specification v 1.1*. [online], Available at: `https://opentracing.io/specification/`, Accessed 2024-01-27.

[16] OpenTelemetry, *OpenTelemetry Specification 1.29.0 - Overview*. [online], Available at: `https://opentelemetry.io/docs/specs/otel/overview`, Accessed 2024-01-27.

[17] SigNoz, *Get Started - How Does SigNoz Work?* [online], Available at: https://signoz.io/docs, Accessed 2024-01-29.

[18] Jaeger, *Architecture v 1.53*. [online], Available at: https://jaegertracing.io/docs/1.53/architecture, Accessed 2024-01-29.

[19] ZipKin, *Architecture Overview*. [online], Available at: https://zipkin.io/pages/architecture, Accessed 2024-01-29.

[20] J. Mace and R. Fonseca, "Universal context propagation for distributed system instrumentation," in *Proceedings of the thirteenth EuroSys conference*, 2018.

[21] J. D. Cavanaugh, "Protocol overhead in ip/atm networks," *ATM Networks*, p. 11, 1994.

[22] D. Gomez Blanco, *Practical OpenTelemetry*, pp. xvi, 20, 40. Berkeley, CA: Apress, 2023.

[23] OpenTelemetry, *What is OpenTelemetry?* [online], Available at: https://opentelemetry.io/docs/what-is-opentelemetry/, Accessed 2024-02-11.

[24] CNCF, *OpenTelemetry becomes a CNCF incubating project*. [online], Available at: https://www.cncf.io/blog/2021/08/26/opentelemetry-becomes-a-cncf-incubating-project/, Accessed 2024-02-12.

[25] OpenTelemetry, *Adopters*. [online], Available at: https://opentelemetry.io/ecosystem/adopters/, Accessed 2024-02-12.

[26] Cloud Native Computing Foundation, *October 2023: where we are with velocity of CNCF, LF, and top 30 open source projects*. [online], Available at https://www.cncf.io/blog/2023/10/27/october-2023-where-we-are-with-velocity-of-cncf-lf-and-top-30-open-source-projects/, Accessed 2024-02-05.

[27] Jaeger, *Client Libraries*. [online], Available at: https://www.jaegertracing.io/docs/1.54/client-libraries, Accessed 2024-02-11.

[28] OpenTelemetry, *Libraries*. [online], Available at: https://opentelemetry.io/docs/concepts/instrumentation/libraries/, Accessed 2024-03-10.

[29] M. Popa and S. Capisizu, "Using binary code instrumentation in computer security," *Informatica Economica*, vol. 17, no. 4, p. 47, 2013.

[30] H. Liu, J. Zhang, H. Shan, M. Li, Y. Chen, X. He, and X. Li, "Jcallgraph: tracing microservices in very large scale container cloud platforms," in *Cloud Computing–CLOUD 2019: 12th International Conference, Held as Part of the Services Conference Federation, SCF 2019, San Diego, CA, USA, June 25–30, 2019, Proceedings 12*, pp. 287–302, Springer, 2019.

[31] G. K. Shuvo, *Tail Based Sampling Framework for Distributed Tracing Using Stream Processing*. 2021.

[32] S. Banda, *Optimizing OpenTelemetry's Span Processor for High Throughput and Low CPU Costs*. [online], Available at: https://doordash.engineering/2021/04/07/optimizing-opentelemetrys-span-processor/, Accessed 2024-02-27.

[33] D. G. Reichelt, S. Kühne, and W. Hasselbring, *Overhead Comparison of OpenTelemetry, inspectIT and Kieker*. CEUR, 2021.

[34] OpenTelemetry, *OTel component performance benchmarks*. [online], Available at: `https://opentelemetry.io/blog/2023/perf-testing/`, Accessed 2024-02-27.

[35] A. Shatnawi, B. Rima, Z. Alshara, G. Darbord, A.-D. Seriai, and C. Bortolaso, *Telemetry of Legacy Web Applications: An Industrial Case Study*. Nov. 2023. working paper or preprint.

[36] A. Detti, L. Funari, and L. Petrucci, "µbench: an open-source factory of benchmark microservice applications," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 3, pp. 968–980, 2023.

A Additional Figures

A.1 Tools and Deployments

Figures A1–A3 exclude outliers for the sake of readability.

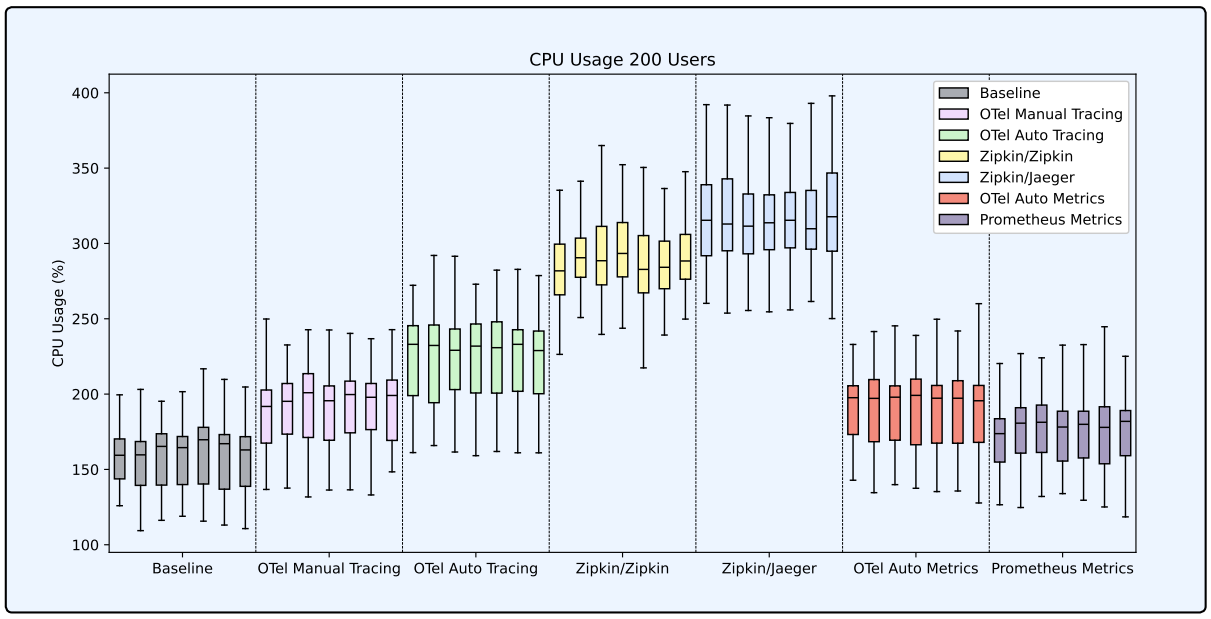


Figure A1: Distribution of CPU usage across all tests, categorized by the various tools and deployment methods used.

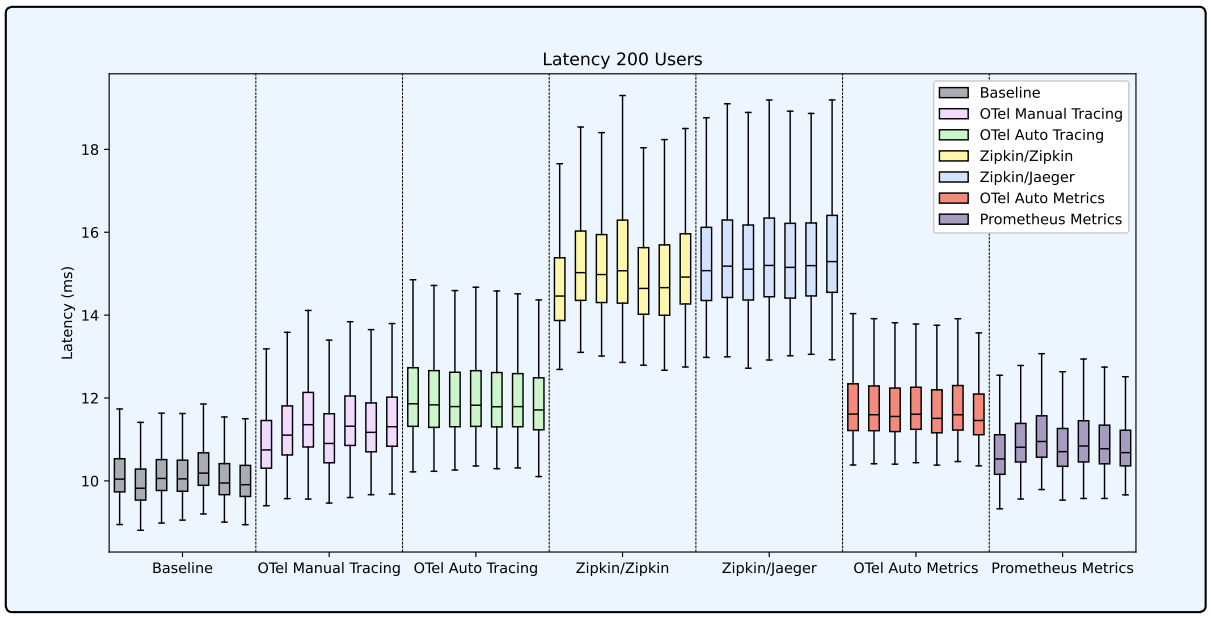


Figure A2: Distribution of latency across all tests, categorized by the various tools and deployment methods used.

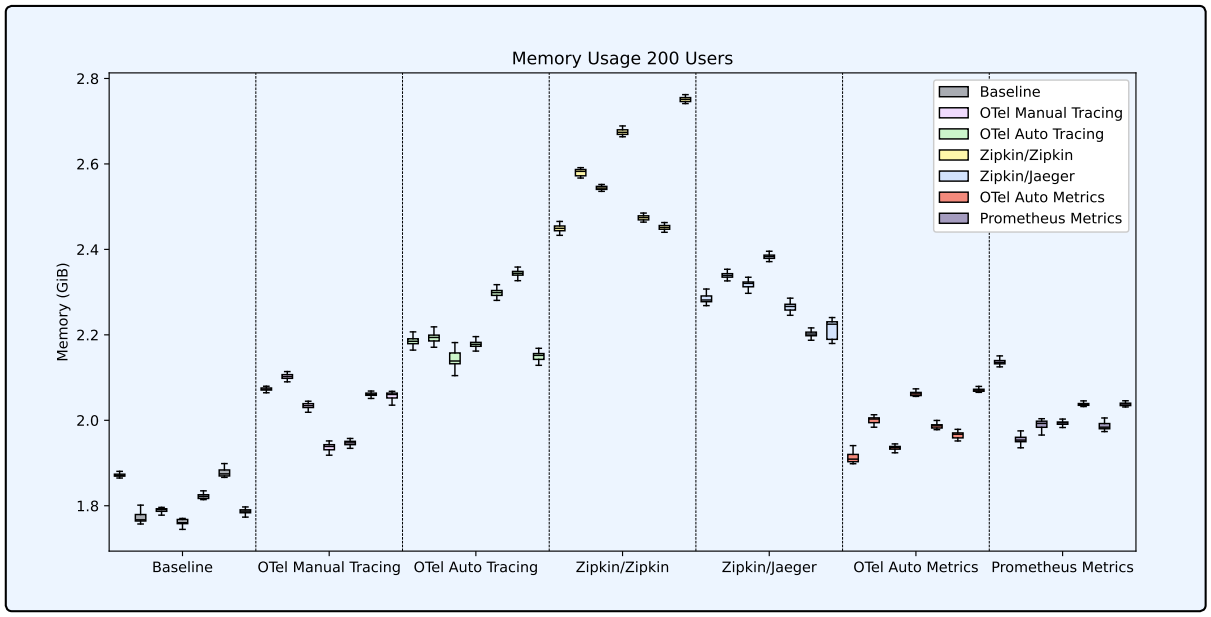


Figure A3: Distribution of memory across all tests, categorized by the various tools and deployment methods used.

A.2 Sampling

Figures A4–A6 exclude outliers for the sake of readability.

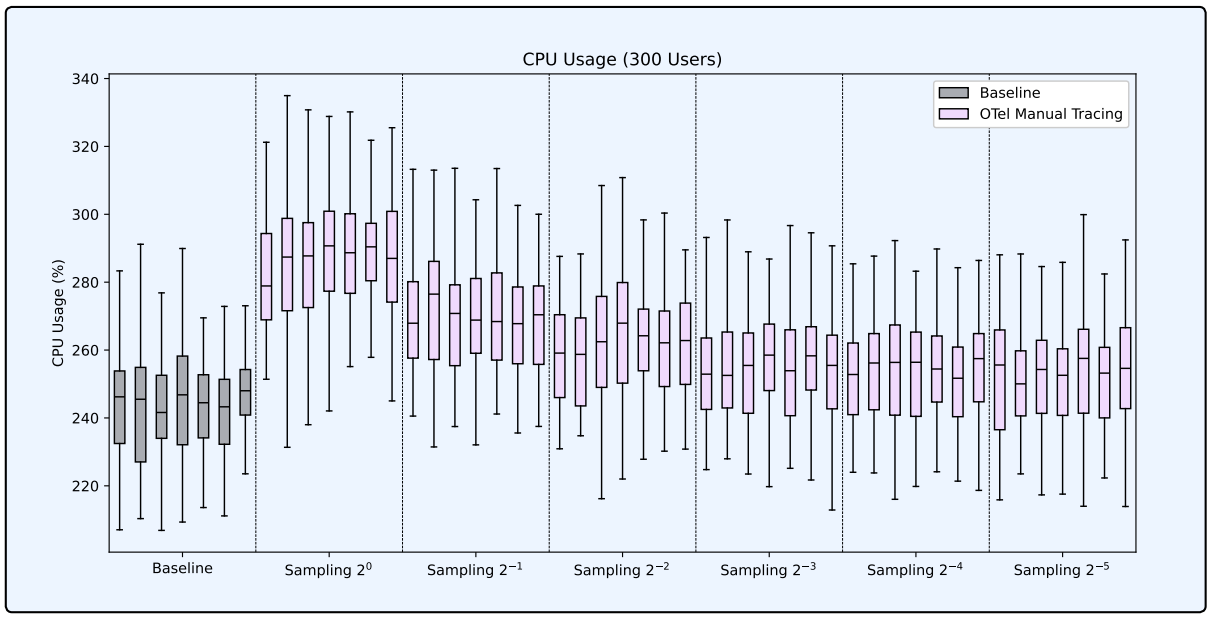


Figure A4: Distribution of CPU usage across all tests, categorized by the baseline and OTel Manual Tracing with various sampling sizes.

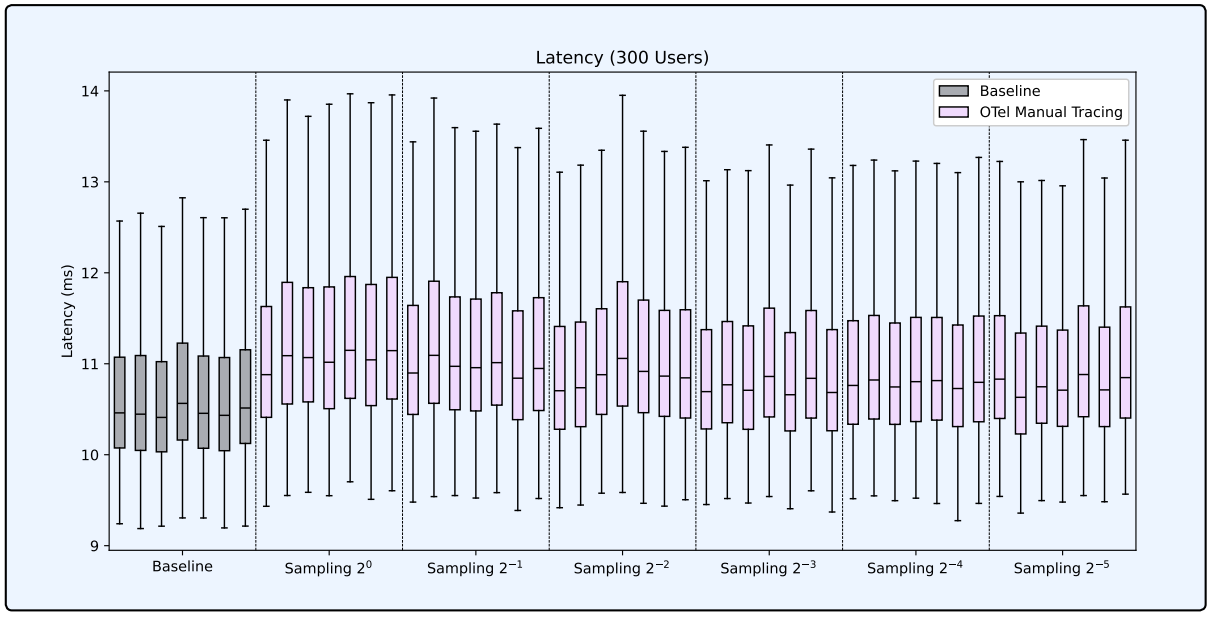


Figure A5: Distribution of latency across all tests, categorized by the baseline and OTel Manual Tracing with various sampling sizes.

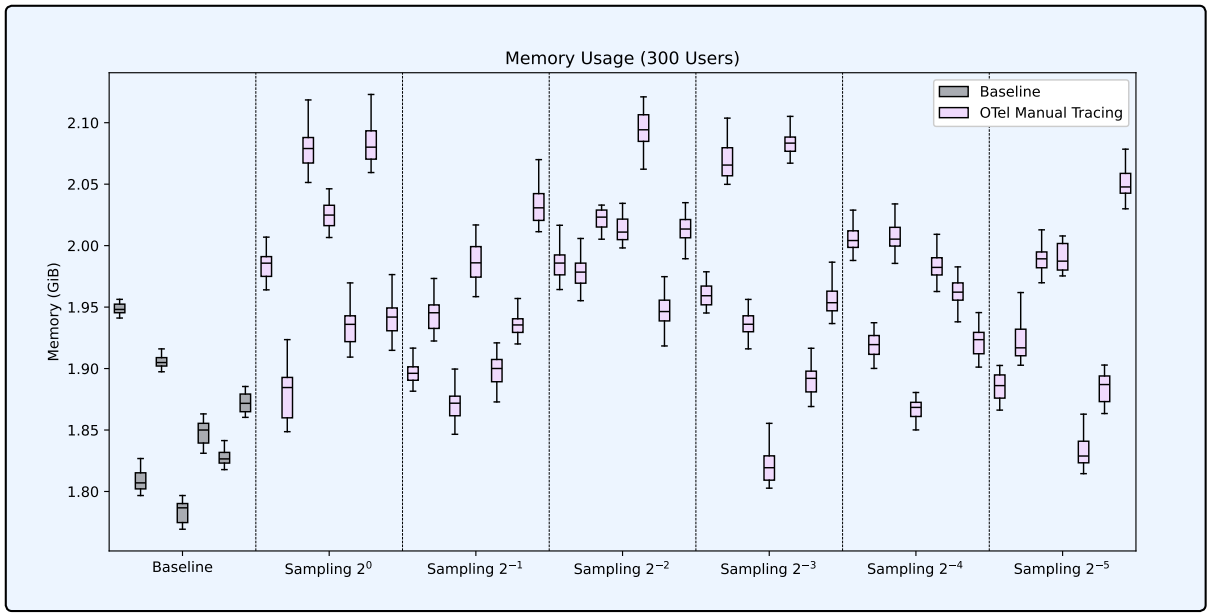


Figure A6: Distribution of memory usage across all tests, categorized by the baseline and OTel Manual Tracing with various sampling sizes.

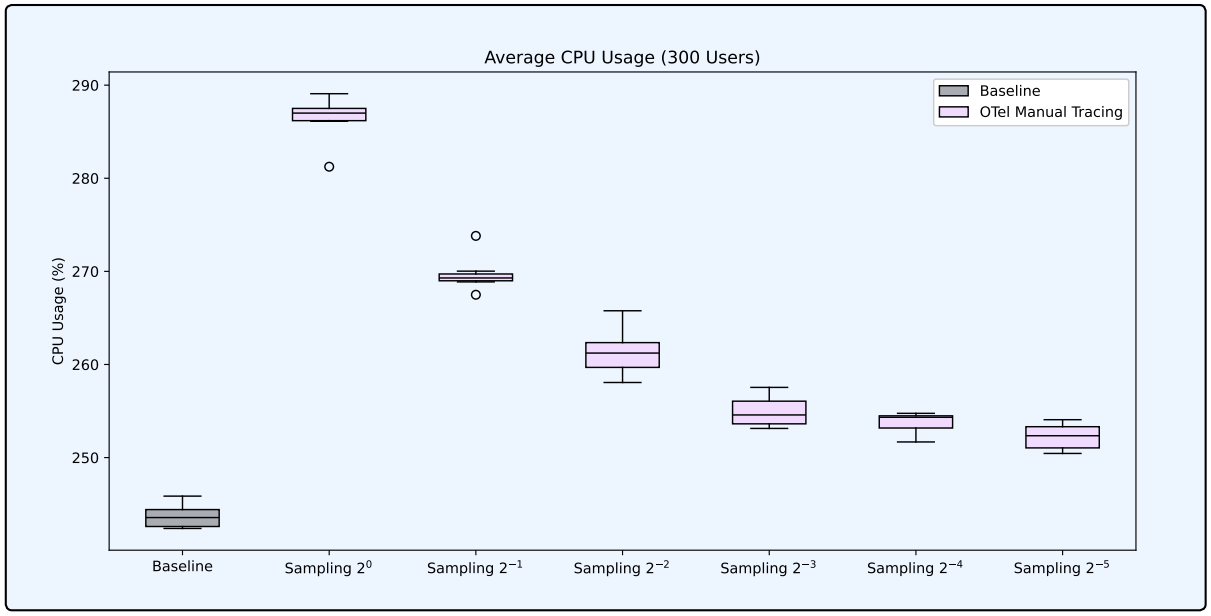


Figure A7: Distribution of average CPU usage, categorized by the baseline and OTel Manual Tracing with various sampling sizes.

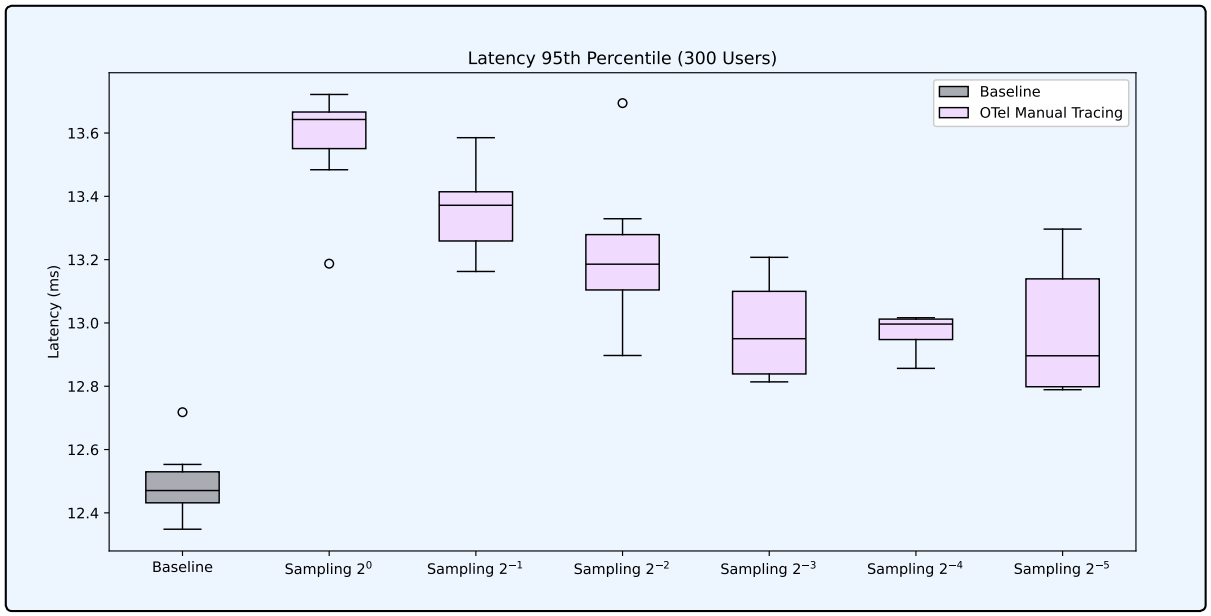


Figure A8: Distribution of 95th percentile latency, categorized by the baseline and OTel Manual Tracing with various sampling sizes.

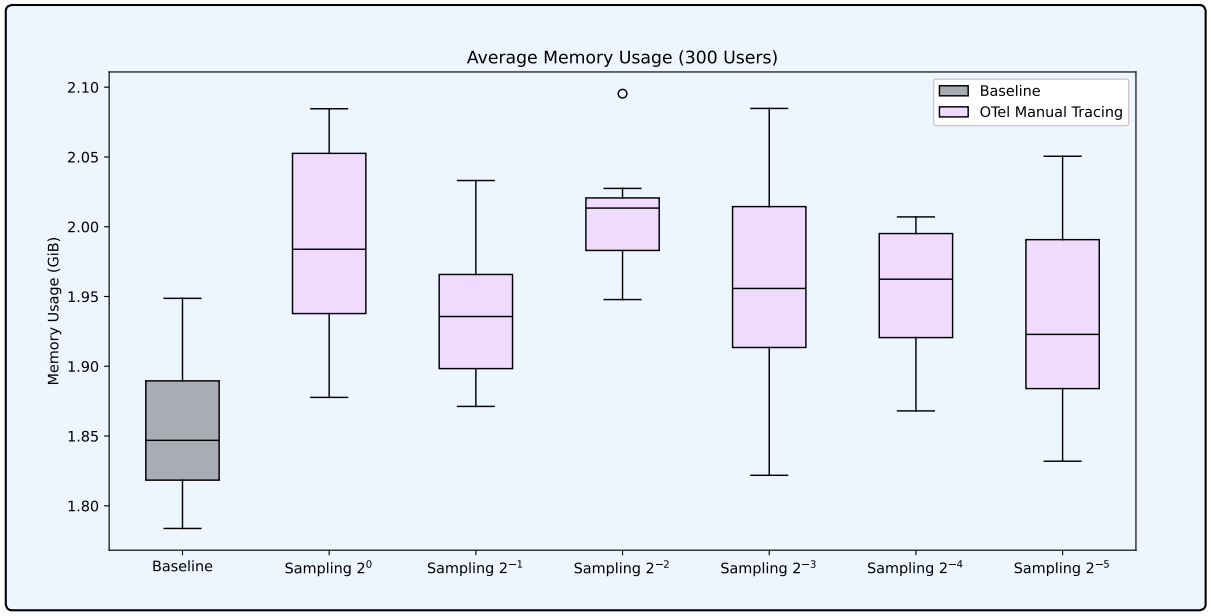


Figure A9: Distribution of average memory usage, categorized by the baseline and OTel Manual Tracing with various sampling sizes.

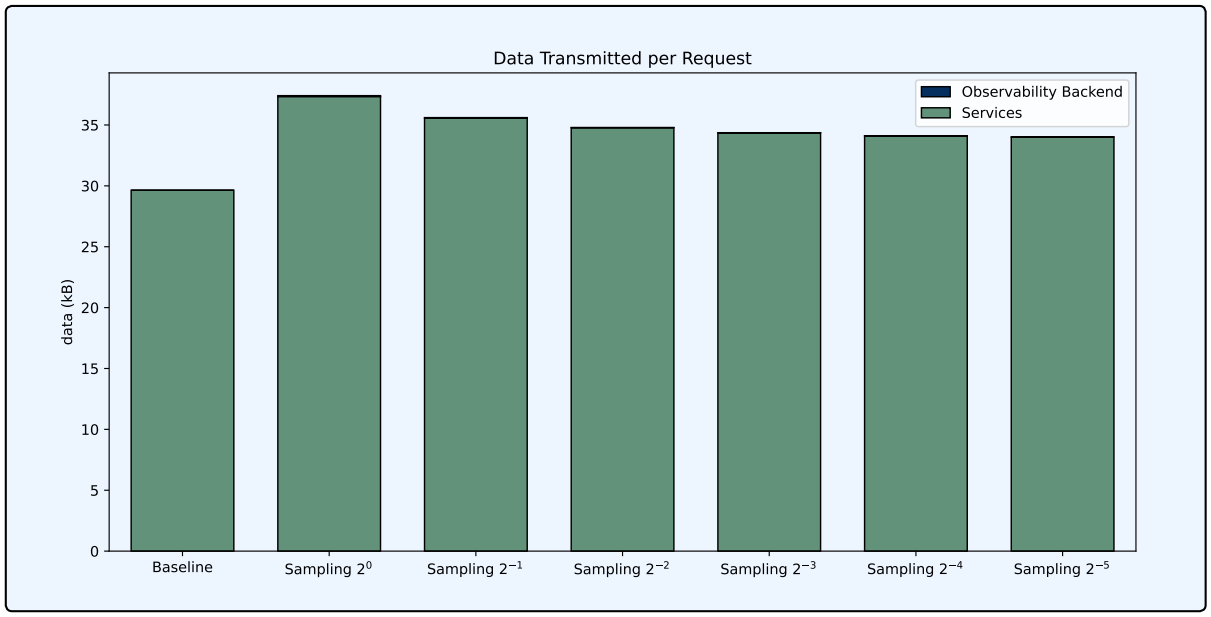


Figure A10: Data transmitted over the network per request, categorized by the baseline and OTel Manual Tracing with various sampling sizes.

A.3 Batching

Figures A11–A13 exclude outliers for the sake of readability.

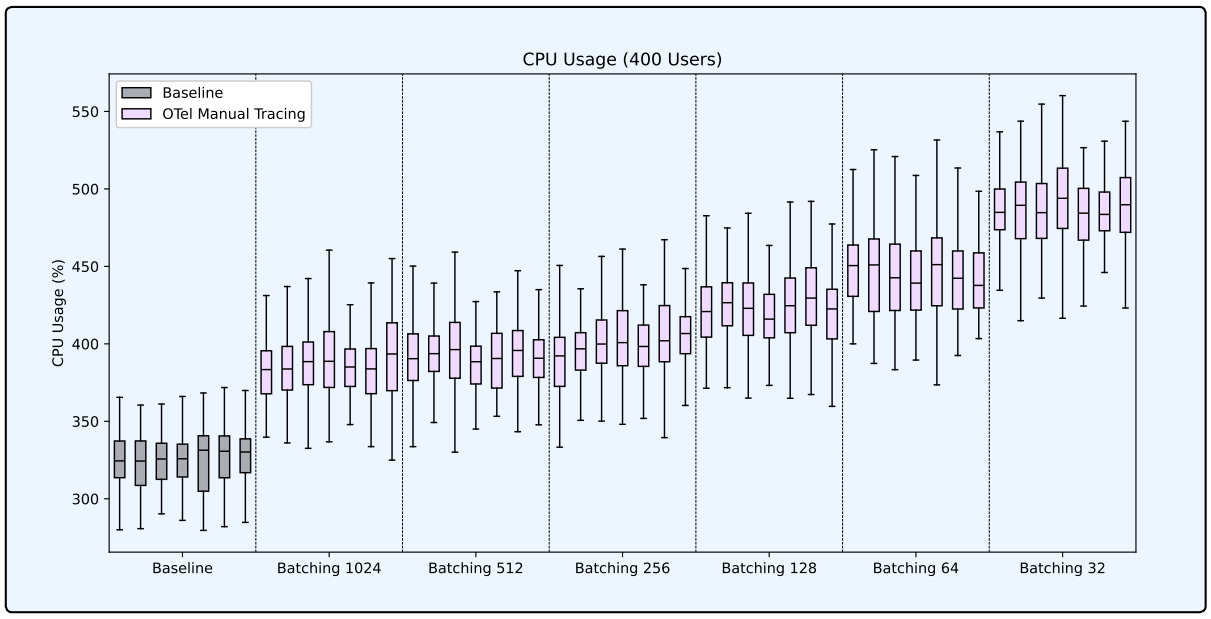


Figure A11: Distribution of CPU usage across all tests, categorized by the baseline and OTel Manual Tracing with various maximum batch sizes.

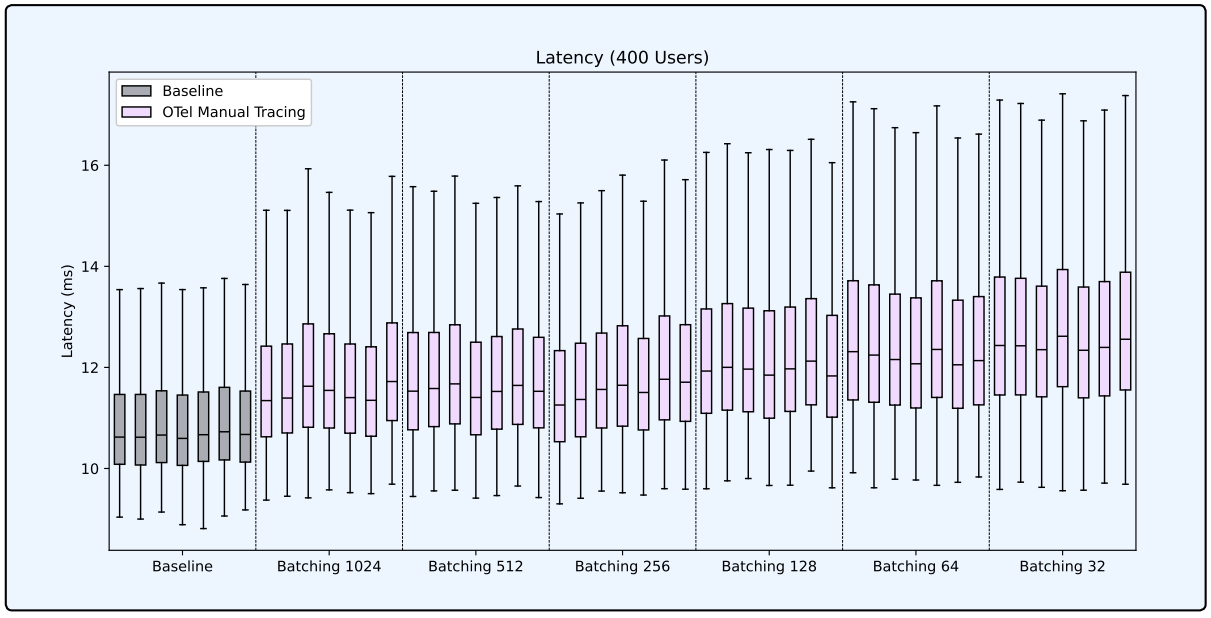


Figure A12: Distribution of latency across all tests, categorized by the baseline and OTel Manual Tracing with various maximum batch sizes

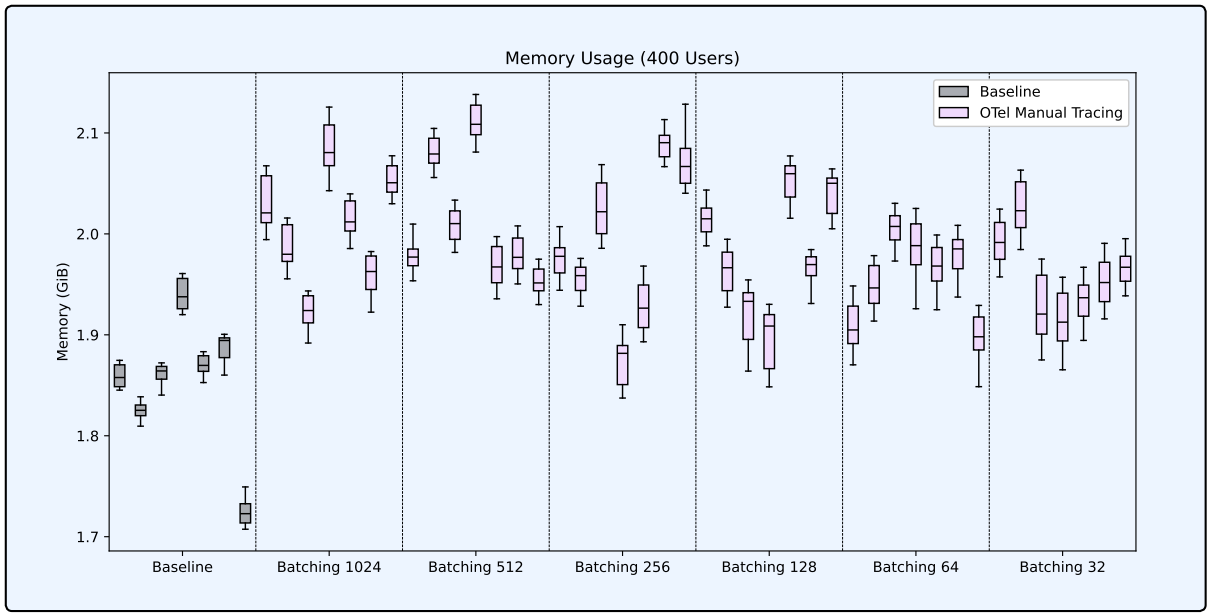


Figure A13: Distribution of memory usage across all tests, categorized by the baseline and OTel Manual Tracing with various maximum batch sizes

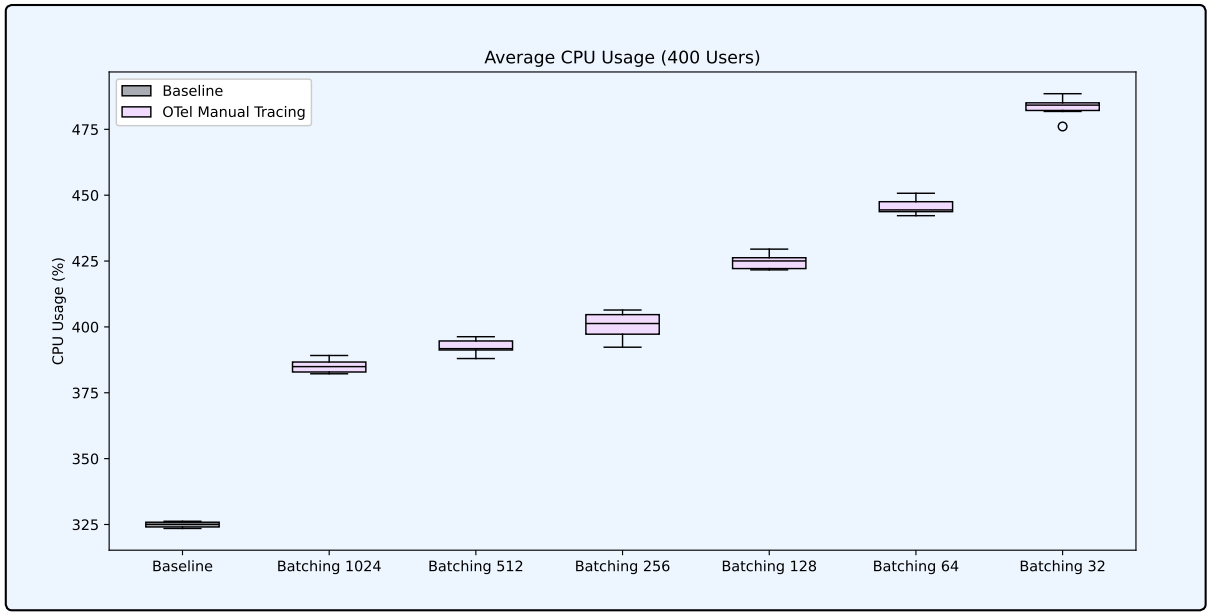


Figure A14: Distribution of average CPU usage, categorized by the baseline and OTel Manual Tracing with various maximum batch sizes.

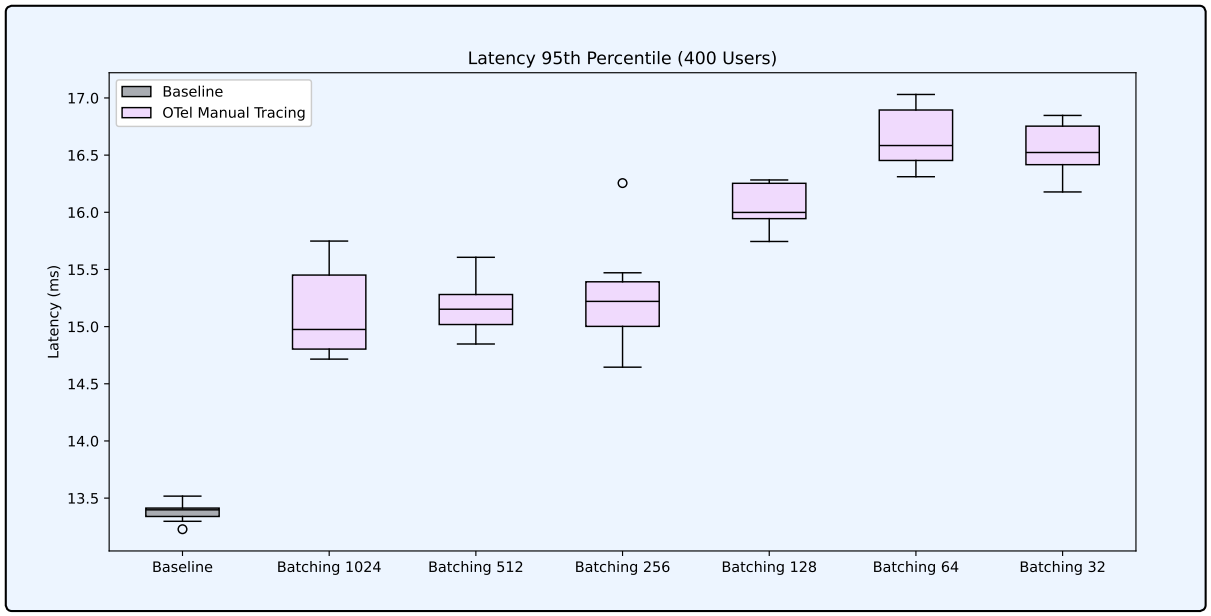


Figure A15: Distribution of 95th percentile latency, categorized by the baseline and OTel Manual Tracing with various sample sizes.

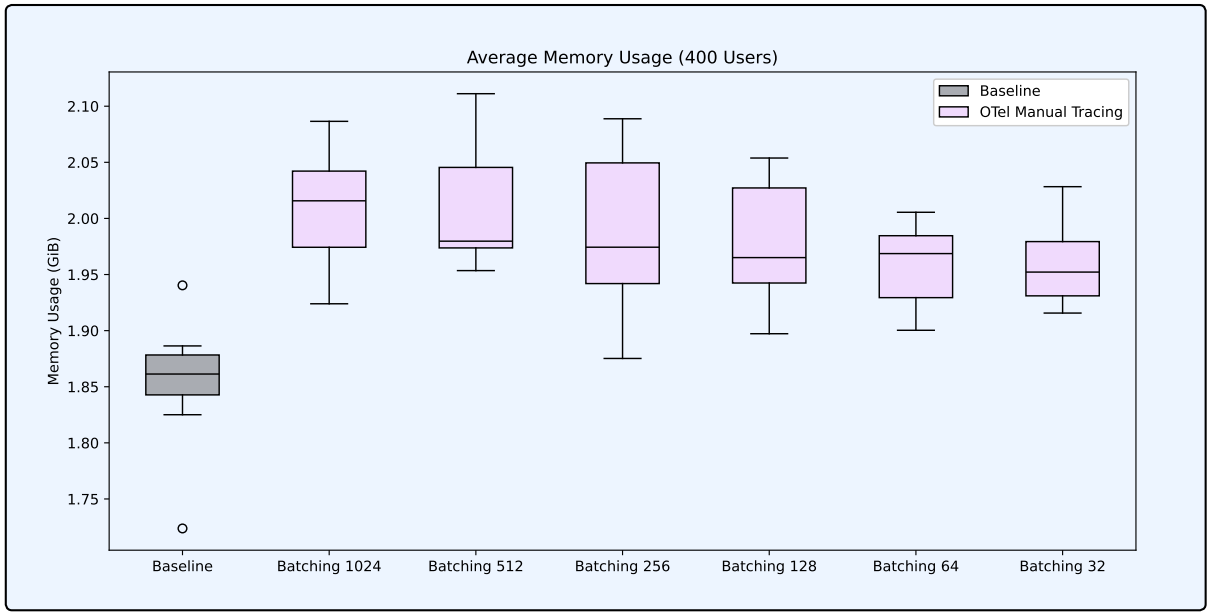


Figure A16: Distribution of average memory usage, categorized by the baseline and OTel Manual Tracing with various sample sizes.